Lowering to AIR - how Mojo kernels reach the Apple GPU

Lowering to AIR

How a Mojo GPU kernel becomes Apple Intermediate Representation, is packaged into a metallib, and is dispatched on an M4 — with no Metal Shading Language anywhere in the pipeline.

Every GPU example in this collection — Fluid, Gray-Scott, Physarum, Boids, the Fern, the Bifurcation diagram, the matmul and STREAM benches — compiles its kernels through one backend: this fork's AIR target inside the Mojo compiler. There is no shader source, no `xcrun metal` run over generated text, no runtime JIT of MSL. The compiler writes the bitcode Apple's driver reads, and the runtime hands it to Metal.

This is the walkthrough of that backend: what AIR is and why it is a harder target than NVPTX or AMDGPU, how the lowering is designed, what it does step by step, the defects met on the way and the gates that now catch each class of them, and what it took to make the result run within a few percent of Apple's own compiler.

> *AIR has no generic address space. A kernel whose stores are in addrspace(1) and whose loads are in addrspace(0) writes correctly and reads zero. No diagnostic, no rejected module, and nothing a verifier objects to — the IR is well-formed, it is simply addressing nothing.*

That sentence, from the findings this port keeps in its oracle repository, is the whole character of the target in miniature. Almost nothing about AIR is documented, most of what goes wrong is silent, and the only specification that exists is what Apple's own compiler emits.

Object backend	`KGEN/lib/Compiler/ObjectCompiler/Target/Air/AirBackend.cpp`, ~2,700 lines; `AirLegality.cpp`, ~1,400 lines
MLIR lowering	`KGEN/lib/KGENToLLVM/Target/Air/AirLowering.cpp`, 328 lines
Target identity	`KGEN/lib/Target/Air/AirTargetProfile.h`, `AirBuiltinRegistry.h`, `AirTraits.cpp`
Runtime	`AsyncRT/lib/MojoBindings/AppleGPURT.cpp`, `AppleGPUMetal.cpp`
Target	`air64_v28-apple-macosx26.0.0` — AIR 2.8, Metal 4.0, LLVM-17-era bitcode
Hardware	Apple M1–M5 profiles; measured on an M4 (10-core) and an M4 Max (32-core)
Reader	Apple's Metal compiler service, reached through `xcrun metallib` and `newComputePipelineState`
Ground truth	`xcrun metal -S -emit-llvm` golden samples, and the `oracles` corpus of released-Mojo output

Where the knowledge comes from

Worth stating plainly, because a backend for an undocumented target invites the wrong assumption. Nothing here was reverse engineered.

There are exactly three sources, and all three are things you can simply run or read:

- **The tree itself.** This is a fork of an Apache-2.0 codebase, so the frontend, the MLIR layer and the optimisation pipeline are ordinary source in this repository. The AIR backend and the Apple GPU runtime are this fork's own code, written into it.
- **A published interface.** `DeviceContext` is a Mojo wrapper over a C ABI whose every symbol is declared in `device_context.mojo`. The *interface* is fully specified in open source; what did not exist was an implementation for Metal, and `AppleGPURT.cpp` is that implementation written against those

declarations.

- **Compilers, run on our own kernels.** `xcrun metal -S -emit-llvm` on our probe kernels shows what Apple's compiler emits; a released Mojo compiler on the same probes shows what its AIR looks like. Comparing our output with theirs on inputs we wrote is differential testing, and it is the only specification an undocumented reader has.

That last one is why "golden sample" and "oracle" appear throughout. They mean a reference *output* for a kernel we wrote, not an artefact taken apart.

One of four

This walkthrough covers the Apple Silicon AIR target. It is one of four concurrent ports, each in its own fork, each aimed at a different reader:

port	target
this one	**Apple Silicon — AIR, Metal 4, M1–M5**
NVIDIA	PTX
Qualcomm	Snapdragon
Mac Pro 2019	AMD Vega II

The NVIDIA, AMD and Snapdragon paths visible in *this* tree are reference material only — each has a fork of its own where its work actually happens, so what is described here is the Apple path and changes to shared lowering are not made on their behalf. Where this document compares AIR with NVPTX or AMDGPU it is describing what those backends do in the shared source, not the state of the sibling ports.

These documents

Chapter	What it covers
1. What AIR is	A frozen reader reached by serialisation, the target identity, and why golden samples are the only specification
2. The pipeline	Every stage from a Mojo `fn` to a `.metallib`, and who owns each one
3. Legalisation	`legalizeModule` step by step: builtins, address spaces, signatures, metadata
4. What went wrong	The defects met on the way, the four gates, and the legality firewall

The chapters, continued:

Chapter	What it covers
5. The runtime	AppleGPURT: unified memory, the address registry, reflection as the argument contract, residency, batching
6. Making it fast	Dispatch at Metal's floor, the optimisation that had to be turned off, the gated unroller, and honest measurement
7. What to understand	The ten things about this backend that are not obvious from watching it work

The shortest possible summary

Mojo elaborates a GPU kernel for NVIDIA: generic pointers, NVPTX address-space numbers, LLVM intrinsics the NVPTX backend consumes in-process. AIR is none of that. It is a *reader* — Apple's frozen LLVM fork inside the Metal compiler service — and it accepts only what Apple's own compiler would have written: every pointer in an explicit address space, every builtin as a type-suffixed `air.*` call, thread identities as trailing kernel parameters carrying metadata, typed pointers in LLVM-17 bitcode, one version stamp that agrees with the triple.

So the backend rewrites the module until it looks Apple-shaped, verifies it while it is still canonical IR, downgrades it, writes it with a bitcode writer built for this reader, and packages it with `xcrun metallib`. The runtime loads that library, asks Metal to reflect the argument contract back, binds buffers by

resolving raw device addresses against a registry it owns, and dispatches inside one compute encoder per batch. Making it fast turned out to mean doing *less*: switching a vectoriser off, unrolling only the loops the source left rolled, and never opening an encoder that a dispatch did not need.

How to read this

Chapters 1 and 2 are the design. Chapter 3 is the mechanism and is the one to keep open next to `AirBackend.cpp`. Chapter 4 is the history of what broke, arranged by the gate that would have caught it, and is where the lessons that generalise to any foreign-reader backend live. Chapters 5 and 6 are the runtime and the performance work; they can be read on their own. Every number in this document was measured on the machines named above, and the bench or finding it came from is named beside it.

1. What AIR is

AIR — Apple Intermediate Representation — is LLVM bitcode with Apple's metadata conventions, written for the LLVM fork inside Apple's Metal compiler. Every `.metallib` is a container of it. When Metal Shading Language is compiled with `xcrun metal`, AIR is what comes out of the front end; when a pipeline state is created at run time, AIR is what the driver's compiler service reads and turns into machine code for the GPU generation it finds itself on.

That is the whole reason this backend exists and the whole reason it is difficult. The GPU's instruction set is not public, the driver compiler does that half of the job, and the only way to reach it is to *serialise* bitcode that the reader will accept.

A frozen reader, reached by serialisation

NVPTX and AMDGPU are in-tree LLVM backends. They consume the optimiser's output in-process, with no serialisation and no version boundary. AIR is reached by writing bitcode for a frozen LLVM fork — the reader is LLVM-18-based and wants the LLVM-17-style typed-pointer bitcode the cooperating writer produces — and that boundary is where every difference between "what modern LLVM emits" and "what a frozen reader understands" turns fatal.

The backend states this in its own words about the optimisation pipeline:

> *AIR has no LLVM codegen target. The TargetMachine (opt pipeline only — emission goes through emitObject/emitBitcode) is built for arm64.*

There is no `AIRTargetMachine`, no instruction selection, no register allocator on this side. The compiler borrows the host's arm64 target so the ordinary LLVM pass pipeline has something to ask questions of, and then throws the arm64 identity away before emission. Everything AIR-specific is done by rewriting IR, not by generating code.

Three consequences follow, and every chapter of this document is about one of them:

1. **Version skew is a correctness problem, not a warning.** A unary `fneg`, a `freeze`, a `poison` constant, a `range` attribute, a GEP no-wrap flag — ordinary modern IR — each crashes the writer or is rejected by the reader. The oracle finding on this quotes the number that makes it systemic: *63% of Modular's GPU test suite is vendor-neutral*, and those tests emit all of the above freely because on the two backends with real coverage they cost nothing. *Every "generic" test is secretly a compatibility test nobody wrote on purpose.*
2. **The machine model is different, and the reader does not say so.** AIR has no generic address space, no 64-bit floats, no three-way compare, no call stack, and a lane that is scalar. Code that violates any of these is mostly not rejected; it is compiled and computes the wrong answer, or the

compiler service dies with a message that names nothing.

3. **The only specification is what Apple's compiler emits.** There is no AIR language reference. Every metadata name, every suffix on every builtin, every version number in this backend was read off a sample produced by `xcrun metal -S -emit-llvm` and confirmed against the corpus of released Mojo output kept in the `oracles` repository.

The target identity is five things

The profile header opens with the observation that took a while to make:

> *The Apple target identity is five separate things that were being carried as scattered literals.*

Fact	Value on this machine	Where it lands in the module
GPU family	`apple-m4`	what the runtime reports; selects capabilities
AIR version	2.8.0	`!air.version`, **and** the `_v28` inside the triple
Metal language version	4.0.0	`!air.language_version`
SDK version	26.0	the `"SDK Version"` module flag
minimum deployment OS	26.0.0	the `macosx26.0.0` part of the triple

Conflating them was the original defect. `metal:4` reads like a Metal language version and actually selects M4 *hardware*. The triple, `air.version` and `air.language_version` each had their own literal with its own comment warning that the other two had to be changed to match. And, as the header goes on:

> *They vary independently. The AIR and Metal versions are properties of the installed TOOLCHAIN, not of the chip: this machine's Metal toolchain emits 2.8/4.0 for an M1 as readily as an M4. The family selects capabilities.*

So a target profile is a pair — a hardware family and a *language profile* — and the alias `apple-m4-metal4` names the pair rather than a sixth architecture. The language profile carries the three version numbers, the SDK and OS stamps, the bitcode writer version, and one more field worth quoting in full because it encodes a mistake the backend has already made once:

> *goldenVerified: False until someone has actually run xcrun metal -S -emit-llvm under this profile and read the numbers off the result. Selecting an unverified profile is refused rather than guessed at — picking the conservative-looking one without measuring is a mistake this backend has already made once, and the module was rejected with no useful error.*

`kMetal4` is golden-verified: the toolchain on this machine, *Apple metal version 32023.830*, emits triple `air64_v28-apple-macosx26.0.0`, AIR 2.8.0, Metal 4.0.0. `kMetal3_2` — the pairing the stdlib's default Apple family carries as `+metal3_2,+air2_7_0` — is deliberately left *unverified*, with zeroed SDK and OS fields, so that selecting it fails loudly instead of stamping invented versions. Nobody has sampled a Metal 3.2 toolchain here.

The disagreement that looks like a bug

The stdlib's default Apple family maps every chip to the same feature string:

```
`#kgen.target<triple = "air64-apple-macosx", `,
`arch = "apple-m4", `,
`features = "+metal3_2,+air2_7_0", `,
```

and the backend stamps 2.8/4.0 regardless. (A parallel `-metal4` family in the same file carries `+metal4_0,+air2_8_0`, which *agrees* with the stamp — and changes nothing, because the stamp never came from the feature string.) That is not an oversight; the profile header explains that the backend *has always been right to*: the installed toolchain emits 2.8/4.0, and the conservative-looking 2.7/3.2 was tried, produced a module the reader rejected, and *taught nothing because the error named nothing*. The feature string selects language features for the stdlib to gate on. What the module is stamped with is what the toolchain writes. The oracle finding that closed this checked every probe in both Apple profiles of the corpus, nine files each, and found AIR 2.8.0 in all eighteen — the two profiles differ only in `air.language_version`, 3.2 versus 4.0.

Two spellings of the triple

The same finding records that Apple's reader accepts two triples:

```
xcrun metal -S -emit-llvm k.metal   ->  target triple = "air64_v28-apple-macosx26.0.0"
released Mojo 1.0.0                 ->  target triple = "air64-apple-macosx26.0.0"
```

This port emits the `_v28` form because the profile builds it from the version fields, which is what keeps the suffix and `!air.version` from drifting apart — they are the same fact stated twice, and a reader that checks both will reject a module where they differ. The optimisation pipeline's *codegen* triple is derived from the same profile for a smaller reason with the same shape: it used to be a literal `arm64-apple-macosx14.2.0` in `AirTraits.h`, contradicting the AIR triple's `macosx26.0.0`, and nobody noticed because *"the host triple" is not where you look when auditing AIR versions*.

Resource limits belong to the profile too

Metal's feature-set limits are module metadata, so the profile carries them even though every Apple family currently agrees:

Module flag	Value
`air.max_device_buffers`	31
`air.max_constant_buffers`	31
`air.max_threadgroup_buffers`	31
`air.max_textures`	128
`air.max_read_write_textures`	8
`air.max_samplers`	16

The first of those is load-bearing for a design decision in chapter 3: a kernel gets thirty-one buffer slots, and the x86-64 fork this backend descends from spent one per captured pointer.

The machine model that matters

Four facts about the hardware shape everything downstream, and each one is invisible from the IR.

A lane is scalar. The backend's comment on why vectorisation is off:

> *An Apple GPU lane is scalar: SIMD is across threads, MSL vectors stop at four wide, and a <16 x float> fmul has no unit to land on.*

Thirty-two threads execute together in a simdgroup. A `float4` is a real type; a `<16 x float>` is sixteen scalar operations wearing a costume, and it carries insert and extract traffic the scalar form never had. Chapter 6 puts a number on it.

There is no generic address space. Device memory is `addrspace(1)`, constant argument memory is `addrspace(2)`, threadgroup memory is `addrspace(3)`, thread-private memory is `addrspace(0)`. A pointer with no address space is not "generic" — it is private, and it addresses nothing the kernel was given. Across every golden sample the port has taken there are *zero* `addrspacecast` instructions and *zero* `addrspace(0)` pointers to anything but allocas.

There is no call stack. Metal kernels are fully inlined regardless of what the IR says. The backend inlines every internal helper *first*, before any legalisation, for reasons chapter 3 quotes.

There is no double. MSL has no 64-bit floating type, and no 128-bit integer. A kernel that uses either is refused by legalisation with an error naming the kernel, rather than silently demoted.

Golden samples: the only specification there is

The project's `STATUS.md` states the technique in two commands:

```
xcrun metal -S -emit-llvm k.metal -o k.ll
xcrun metal -x ir -c k.ll -o k.air
```

Write the smallest MSL kernel that exercises the construct in question — our kernel, Apple's shipped compiler — read the textual AIR its front end emits, and match that shape exactly: the metadata tuple order, the builtin suffix, the address space on each parameter. Then inspect the result with the shipped `air-objdump`, `air-readobj`, `air-nm`, `air-opt` and `air-link` tools.

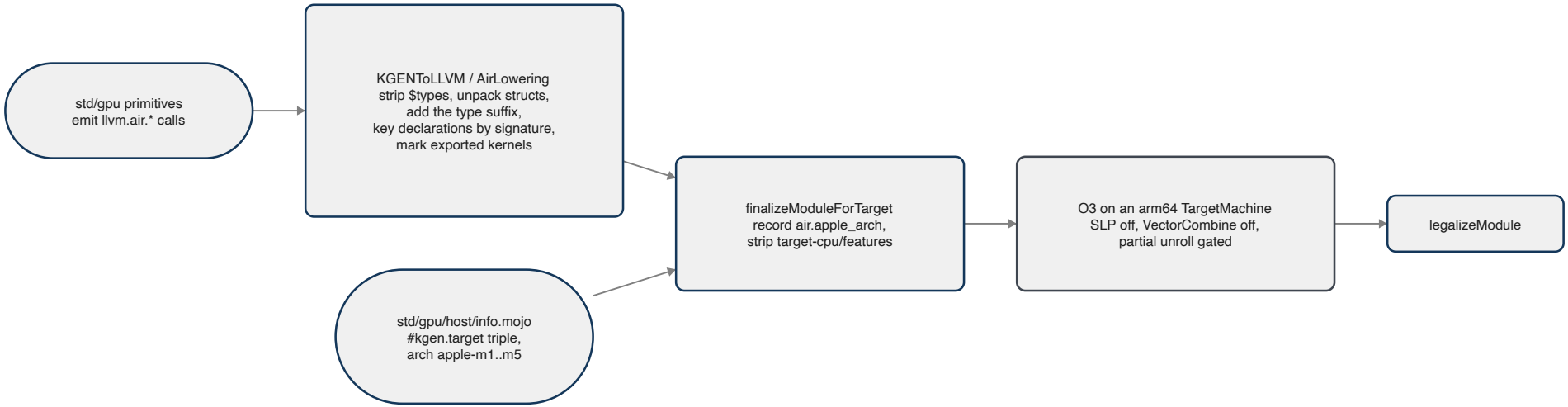
The most useful output of a golden sample is inverted — *what the reference never emits*. For Apple that list is `addrspacecast`, `addrspace(0)`, and the native `sitofp` / `uitofp` / `fptosi` / `fptoui` casts. Every one of those was a real defect when this backend emitted it, and, as the diagnostics finding

puts it, *each cost a separate day to find individually*.

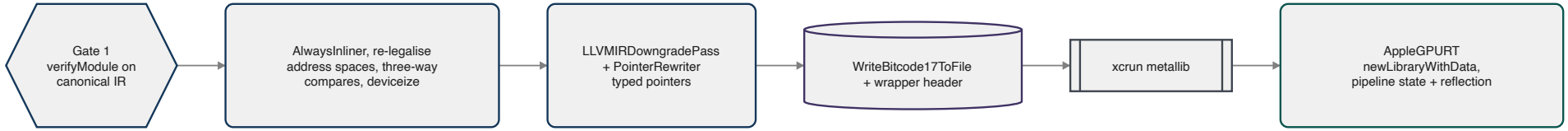
The second oracle is the `oracles` repository beside this one: the AIR a released Mojo compiler emits for a corpus of probe kernels we wrote, on the `apple-m4` and `apple-m4-metal4` targets. Where Apple's compiler shows what the reader was designed for, the released compiler shows what a *Mojo* kernel looks like when it arrives correctly — the same generic-pointer, NVPTX-numbered input this backend receives, already legalised by the people who own the frontend. Both are consulted throughout, and when they disagree (the two triple spellings, the scalar `llvm.fma.f32` Apple never emits but the released compiler does) the reader's acceptance of both is the fact recorded.

2. The pipeline

A kernel passes through four owners on the way to the GPU: the Mojo standard library, which names the Apple builtins; the MLIR-to-LLVM lowering, which turns those names into declarations; the object backend, which does everything AIR-specific to the LLVM module and writes the artefact; and the runtime, which loads it. The boundaries between them are deliberate, and two of them were drawn after a defect showed where the old boundary was wrong.



The back half turns that module into bytes Apple's reader accepts, and hands them to the runtime:



Stage 1: the standard library names the builtins

Mojo's GPU primitives are written per vendor. On the Apple path, `thread_idx`, `block_idx`, `block_dim`, `grid_dim`, `lane_id`, the warp shuffles and reductions, `barrier()`, and the simdgroup matrix operations each become a call to a function that does not exist: a name of the form `llvm.air.<builtin>` or `llvm.air.<builtin>.<dim>`. Twenty-five distinct stems appear across the tree today, from `llvm.air.thread_position_in_threadgroup.` and `llvm.air.threadgroup_position_in_grid.` (with a dimension appended) through `llvm.air.simd_shuffle_xor`, `llvm.air.simd_sum`, `llvm.air.simd_ballot.i32`, `llvm.air.wg.barrier`, `llvm.air.simdgroup.barrier`, a handful of transcendental maths, and the three `simdgroup_matrix_*_multiply_accumulate` forms the M4 and M5 MMA paths use.

These are *not* LLVM intrinsics. The AIR lowering header is explicit:

> *Owns the lowering of llvm.air.* builtin "intrinsics": those names are not real LLVM intrinsics, so they lower to calls of air.*-named external functions which the AIR backend later converts to kernel parameters / mangled AIR runtime calls.*

The distinction matters because LLVM will not mangle a name it does not own. `llvm.fma` becomes `llvm.fma.v4f32` automatically; `llvm.air.simd_sum` does not become anything, so every operand type would resolve to *one* declaration, and the second signature would assert during MLIR-to-LLVM translation — *"Calling a function with a bad signature!"*, a hard compiler crash in a stack naming no user code. That was a real failure class, closed at the declaration site rather than enumerated at each Mojo call site.

Stage 2: the MLIR lowering owns declaration creation

`AirLowering.cpp` is short — 328 lines — and it has exactly one job: turn each `llvm.air.*` call into a correctly typed declaration of a real `air.*` symbol. Four things happen to every such call:

1. **The declaration is created under a tag keyed to its signature.** The symbol the lowering mints is `air.<stem>$<hash of the exact FunctionType>`. The real AIR name is derived from the operand types here, and the tag *only ever has to be unique, never correct* — the object backend's `airStem` drops everything from `$` before the final `air.*` name is assembled, and `stripAirSignatureTags` sweeps what survives.
2. **Struct operands are unpacked.** KGEN packs multi-operand intrinsic arguments into a struct; AIR runtime functions take flat scalars. A Mojo `Bool` arrives as `{i1, [15 x i8]}`, and the trailing byte-array padding has to be skipped rather than flattened into a spurious parameter.

3. **The type suffix is added**, from the builtin registry: `air.simd_shuffle_xor.u.i32`, `.f32`, `.f16`, each read off a golden MSL probe.
4. **The function type is the identity, not the name.** Any symbol found is checked against the calling FunctionType, so two kernels using the same builtin at different payload types get two declarations and no collision — a hash collision surfaces as a located diagnostic rather than a miscompile.

One decision in that file deserves its own paragraph because it is a guess made carefully. AIR carries *separate* signed and unsigned integer symbols — `air.simd_sum.s.i32` and `air.simd_sum.u.i32` both exist — and by the time the lowering runs, the LLVM dialect's integers are signless and the stdlib has emitted a bare stem. The signedness is simply not in scope. The lowering chooses `.u.` and then says exactly where that is sound:

> *sum, product, prefix sums — two's-complement add/multiply are bit-identical signed vs unsigned. SAFE. shuffles — a lane move does not interpret the payload. SAFE. min, max — genuinely different. min(-1, 5) is -1 signed and 5 unsigned. NOT SAFE, and rejected below rather than guessed at.*

Today the stdlib never emits `llvm.air.simd_min` or `simd_max`, so the rejection is a tripwire: wiring them up will fail at compile time instead of silently computing the wrong reduction. There is no 64-bit case at all, since MSL rejects simdgroup operations on 64-bit types outright.

Why this pass is module-scoped

The header records the triage finding that put it there:

> *The conversion creates module-level function declarations, so it MUST run in the module-scoped, single-threaded LowerGlobalPOPToLLVM pass — doing it from the per-function LowerPOPToLLVM pass raced sibling function conversions on the symbol table (triage finding: duplicate declarations / crashes when several kernels share a builtin).*

Why the object backend refuses to do this job

`AirBackend.cpp` used to contain a fallback that rewrote any surviving `llvm.air.*` shim itself. It now *verifies* that none survived, and fails hard if one did. The comment on that decision is the clearest statement of the ownership rule in the tree:

> *An object backend should not be reconstructing operation semantics from MLIR in the first place.*

The fallback had never fired on real input, because it ran before the lowering pipeline had created anything for it to find; and had it ever fired, it would have looked a declaration up by *name* and reintroduced precisely the bare-symbol type collision the MLIR lowering exists to prevent — *a dead fallback waiting to become live after an unrelated pipeline change*. Reaching the object backend with a shim means the target hooks did not run, and that deserves a located failure, not a silent recovery.

Stage 3: the object backend

`AirBackend` is a `TargetBackend` with three properties that shape everything else: it is an *offload* target, it is *not* a base target, and it splits the module `PerExported` — one exported kernel per emitted module, which is why every retained artefact is named per kernel.

Before optimisation: record the arch, then strip the host attributes

`finalizeModuleForTarget` runs once the TargetMachine exists and before the optimiser. KGEN's LLVM lowering stamps `target-cpu`, `target-features` and `tune-cpu` on every lowered function from the `kgen.target` attribute. For an AIR target the feature string is upstream's `+metal3_2,+air2_7_0`, and the TargetMachine is arm64; the first optimiser pass that asks for a subtarget builds an AArch64 subtarget *from that attribute*, and LLVM's feature parser answered, on every GPU compile:

```
'+air2_7_0' is not a recognized feature for this target (ignoring feature)
```

Harmless to the output — the scrub before emission strips the same attributes — but noise on every build, and *a warning nobody reads is the kind that hides the one that matters*. So the attributes come off here, declarations included, because the subtarget query is made about callees as well as bodies.

The order of the two lines in that function is the fix for defect D21:

> *legalizeModule needs the arch these attributes carry, and it runs AFTER this. Record it first (kAppleArchMD), then strip — the order whose absence was D21.*

The first version of the warning fix stripped the attribute and left legalisation to read an empty arch. Every GPU program in the tree then failed with `no Apple AIR target profile for arch ''`, and — chapter 4 tells the rest — the change had been pushed as verified. The arch now travels as named module metadata, `air.apple_arch`, which legalisation reads and then forgets before the artefact is written.

The shared O3 pipeline, with hooks

The optimisation pipeline is Modular's, shared with every target, and this port threads the backend through it so a target can decline a pass. Five virtuals on `TargetBackend` exist for that: `wantsVectorization`, `wantsVectorCombine`, `wantsPartialUnrolling`, `unrollBodyLimit` and `unrollSingleBlockOnly`. On AIR the first two answer no and the third yes, for reasons that are the substance of chapter 6. Each is an environment knob as well, and each prints an `[air-knobs]` line to stderr when it is set, for a reason chapter 4 explains under the compile cache.

Knob	Default	What it controls
`APPLEGPU_AIR_VECTORIZE`	off	SLP vectorisation in the O3 pipeline
`APPLEGPU_AIR_VECTOR_COMBINE`	off	the VectorCombine pass
`APPLEGPU_AIR_UNROLL`	on	the gated partial unroller
`APPLEGPU_AIR_UNROLL_LIMIT`	128	loop body size the gate admits
`APPLEGPU_AIR_UNROLL_PARTIAL_THRESHOLD`	1024	LLVM's partial-unroll threshold
`APPLEGPU_AIR_UNROLL_GATE`	single-block	`wide` also admits threadgroup-only multi-block loops
`APPLEGPU_AIR_UNROLL_PARTIAL`	off	the ungated unroller inside `emitObject`, after legalisation
`APPLEGPU_AIR_KEEP_LOOP_MD`	off	keep `!llvm.loop` metadata in the artefact
`APPLEGPU_AIR_OPT_LEVEL`	3	the device pipeline's optimisation level
`APPLEGPU_KEEP_AIR=<dir>`	unset	retain `.pre.ll`, `.post.ll`, `.air` and `.metallib` per kernel
`APPLEGPU_AIR_XFORMS`	all off	the table-driven legality transforms

Knobs are read from the environment first, then from the first readable of `APPLEGPU_XFORMS_FILE`, `~/.applegpu-xforms`, or `/tmp/applegpu-xforms.conf` (the last only if the current user owns it — world-writable `/tmp` is not a place to take instructions from), so an experiment can be pinned for a whole suite run without editing every command.

Inside `emitObject`: legalise, verify, then the downgrade

`emitObject` is the whole second half, in this order: `legalizeModule` itself first — chapter 3 walks it, and its own first act is to inline every internal helper — then Gate 1, and only then a short pass sequence in front of the downgrade. The order is the point:

1. **AlwaysInlinerPass first.** Whatever is still un-inlined is forced into its caller now, because the passes below re-legalise, and code this pass pulls in from a callee has never been through them.
2. **The ungated partial unroller**, behind `APPLEGPU_AIR_UNROLL_PARTIAL` — for a comparison run at this position rather than through the gated one in the main pipeline.
3. **Break wide vector arithmetic**, behind `APPLEGPU_AIR_SCALARIZE_WIDE_VECTORS`: a scalariser with `ScalarizeMinBits=128` fragments anything wider than `float4` while leaving loads and stores alone — *vector memory access is genuinely wide on this hardware, and splitting it would cost*.
4. **Re-legalise address spaces** now that inlining has run, because code the inliner pulled in from a callee has never been through the pass: *On AIR that is not a missed optimisation, it is a wrong answer.*
5. **Lower three-way compares and deviceize captured pointers again**, for the same reason.

Then the downgrade itself.

Gate 1 sits on canonical IR

`llvm::verifyModule` runs immediately after legalisation and *before* the downgrade. The first placement was after the downgrade and was wrong:

> *PointerRewriter deliberately rewrites pointers to TypedPointerType for the LLVM-17 writer and (per MojoMacX64's triage) drops the lifetime-on-alloca exemption on purpose, so a module that is correct for its purpose fails verification there: 23 spurious "llvm.lifetime.start/end can only be used on alloca or poison" against 4 real findings. A gate that cries wolf gets switched off.*

A rejected module is *kept*: the gate fires before any retained-artefact hatch runs, and without that a rejection would leave nothing behind — *and the module IS the diagnosis*.

Downgrade, write, package

The emission machinery is Apple's published Metal skeleton, filled in:

> *LLVMIRDowngradePass is the published MetalAIRPass skeleton today; our legalizeModule above supplies the body that upstream leaves out.*

`LLVMIRDowngradePass` folds modern constructs into what the frozen reader knows. `PointerRewriter` turns every opaque pointer into a `TypedPointerType`, recording the retyped function signatures in its own side table — the source of a subtle rule chapter 4 covers. `WriteBitcode17ToFile` is the cooperating writer, described in its own header as *"for writing Metal bitcode"*; it emits typed POINTER records and the bitcode wrapper header itself. Between the downgrade and the write the backend takes a last look at the module's declared external symbols — a check, not a listing: any symbol the AIR contract does not know, or any declaration nothing defines, fails the build there — because an unresolved external is *not* rejected by `metallib`; it survives packaging and kills the compiler service at pipeline creation with `XPC_ERROR_CONNECTION_INTERRUPTED`, naming nothing at all.

The `.air` is copied to the retained-artefact directory *before* packaging. The comment records why that ordering is not cosmetic:

> *When metallib rejects the bitcode ("Unexpected bitcode file!") the rejected file is exactly what you need, and copying it only on success meant it was deleted at the one moment it mattered.*

Packaging is `xcrun metallib`, looked up on `PATH` first and falling back to `/usr/bin/xcrun` — under Bazel the fallback is what fires, because there *is* no `PATH`: `rules_mojo` pins the compile action's environment to `PATH=/dev/null` for hermeticity, confirmed with `bazel aquery`, and `--action_env=PATH` cannot override a rule's own environment. The system stub resolves the active toolchain itself through `DEVELOPER_DIR`, which the rule does pass through. The failure that produced is worth keeping in mind whenever the build stops with what looks like a compiler error: *the AIR bitcode is generated FIRST and only packaging fails, so the whole codegen path can be working and the build still stops*.

Which view is which

Three views of the module exist, and only one of them is the artefact:

- `--emit=bitcode` shows the module *as the pipeline produced it*, deliberately not legalised. Running `legalizeModule` there would also corrupt any module that later went through `emitObject`, because legalisation is not idempotent: each run appends another `!air.kernel` operand per kernel.

- `--emit=asm` is the legalised textual IR — the debugging view. It is a sidecar, not the module that went through `emitObject`'s post-O3 passes, so it is the wrong thing to read when the question is what a knob did.
- `APPLEGPU_KEEP_AIR=<dir>` retains the real thing, four files per kernel: `<kernel>.pre.ll` as legalised, `<kernel>.post.ll` after the downgrade, `<kernel>.air` as written, `<kernel>.metallib` as packaged. Every experiment in chapter 6 was read off those.

Stage 4: the runtime

The metallib bytes reach `AppleGPURT` through Modular's device ABI, and from there Metal's own API does the last step: `newLibraryWithData`, `newFunctionWithName`, `newComputePipelineStateWithFunction:options:reflection:`. The reflection result is the argument contract the launch path binds against. Chapter 5 is that half of the story.

3. Legalisation

`legalizeModule` is the body Apple's published `MetalAIRPass` skeleton leaves out. It is about two hundred and forty lines of calls in a fixed order, and most of the order encodes a defect that happened when it was different. This chapter walks it top to bottom.

0. One module, one target

The first thing the function does is resolve the target profile, from the `air.apple_arch` metadata that `finalizeModuleForTarget` recorded. The comment insists on doing it first because *the triple and every version stamp* derive from it, and insists on doing it once:

> *One module, one target. Taking the first function's arch silently compiles ...*

— the rest of that sentence being what a module with two archs would do. Every function in the module must agree, or legalisation refuses.

1. Inline every helper, before anything else

`inlineInternalHelpers` runs before a single AIR-specific rewrite. AIR has no call stack and Metal kernels are fully inlined regardless, so it costs nothing; the reason it runs *first* is a list:

> *— deviceizeCapturedPointers never saw code that inlining brought in later, leaving device pointers generic (reads returned zero, silently); — propagatePointerAS retyped a defined callee's parameter, which leaves the enclosing FunctionType behind and the module invalid; — a kernel using threadgroup memory THROUGH a helper could not have the global rewritten to a parameter, because the argument does not exist inside the callee. Every one of those is "the code moved after I legalised it". Inline first and the question does not arise.*

2. Three constructs the optimiser invents and AIR lacks

Three small lowerings run next, each for an instruction that never appears in Mojo source and that LLVM's InstCombine synthesises from ordinary code:

- **lowerVectorFMA** — a vector `llvm.fma` must be `air.fma.<ty>`. Apple's compiler emits `air.fma.v4f32`; the scalar `llvm.fma.f32` is accepted as it is. The split is by width.
- **lowerMaskBitcasts** — InstCombine's idiom for "are any lanes set" is `bitcast <4 x i1> to i4` followed by `icmp eq i4 %b, 0`. An `i4` is not a register width any GPU implements. The pair is lowered to an OR reduction over the lanes so the odd-width integer is never materialised — and only when every user is a comparison against zero, since anything else risks silently wrong code.
- **lowerThreeWayCompares** — `llvm.scmp` and `llvm.ucmp`, which AIR has no instruction and no runtime function for. Othello's Monte-Carlo tree search kernel reached the reader as the undefined symbol `llvm.scmp.i32.i64`, *naming neither the kernel nor anything in the source*. Both expand into selects.

3. The transform table

`Air::applyTransforms` applies a table of optional rewrites, all off by default and selectable with `APPLEGPU_AIR_XFORMS=name=on` or `all=on`: `rename-llvm-intrinsics`, `split-i64-shuffle`, `guard-nan-minmax`, `volatile-loop-loads`. They come from the out-of-tree LLVM AIR backend's experience rather than from a defect measured here, so they are kept as *evidence, not specification* — available for a comparison, never applied unasked. Chapter 4 has the rule table that goes with them.

4. Builtins become AIR runtime calls

`mangleAirOps` finishes what the MLIR lowering began. The builtin registry — `AirBuiltinRegistry.h`, *the shared source of truth for AIR builtin families, signature classes, payload domains, convergence, and type suffixes* — lists every family the backend will construct a declaration for. A family absent from the table is not accepted merely because its name starts with `air.`.

Signature class	Families
Barrier	`air.wg.barrier`, `air.simdgroup.barrier`
Unary	`air.sin`, `air.cos`, `air.tan`, `air.exp`, `air.exp2`, `air.exp10`, `air.log`, `air.log2`, `air.log10`, `air.sqrt`, `air.rsqrt`, `air.recip`, `air.fabs`, `air.floor`, `air.ceil`, `air.rint`, `air.round`, `air.trunc`, `air.frac`, the inverse trig and hyperbolic functions, and `air.simd_sum`, `air.simd_product`, `air.simd_min`, `air.simd_max`, the two `air.simd_prefix_*_sum` scans

The signature classes, continued:

Signature class	Families
Binary	`air.fmin`, `air.fmax`, `air.fmod`, `air.pow`, `air.powr`, `air.divide`, `air.copysign`
Ternary	`air.fma`
Shuffle	`air.simd_shuffle`, `air.simd_shuffle_up`, `air.simd_shuffle_down`, `air.simd_shuffle_xor`
Ballot	`air.simd_ballot.i32`

Each family says whether it carries a type suffix, what payload domain it accepts, and whether it is *convergent* — the barrier, shuffle, reduction and ballot families are, and `applyAirCallAttributes` stamps the attribute so no later pass duplicates or sinks a rendezvous. Two deliberate irregularities are in the experiment log: `air.simd_ballot.i32` is a *pre-mangled* ABI name, because its operand is `i1` and its result `i32`, so deriving a suffix from operand zero would be wrong; and SIMD payloads are limited to the scalar domains the current Metal path supports, so the earlier backend's habit of manufacturing a `.u.i64` spelling is gone.

Then `stripAirSignatureTags` removes any `$types` tag that survived, and `eraseDeadIntrinsicDeclarations` sweeps out every `llvm.*` declaration nothing calls — as a sweep, not per intrinsic, because *the next lowering will strand another one. Ours did, immediately.*

5. Casts

`lowerIntFloatConverts` replaces every integer-to-float and float-to-integer cast with a call, because Apple's compiler emits none of the native forms — ever. Leaving them native does not fail cleanly:

> *metallib accepts the module and the Metal compiler SERVICE dies at pipeline creation with XPC_ERROR_CONNECTION_INTERRUPTED, naming nothing.*

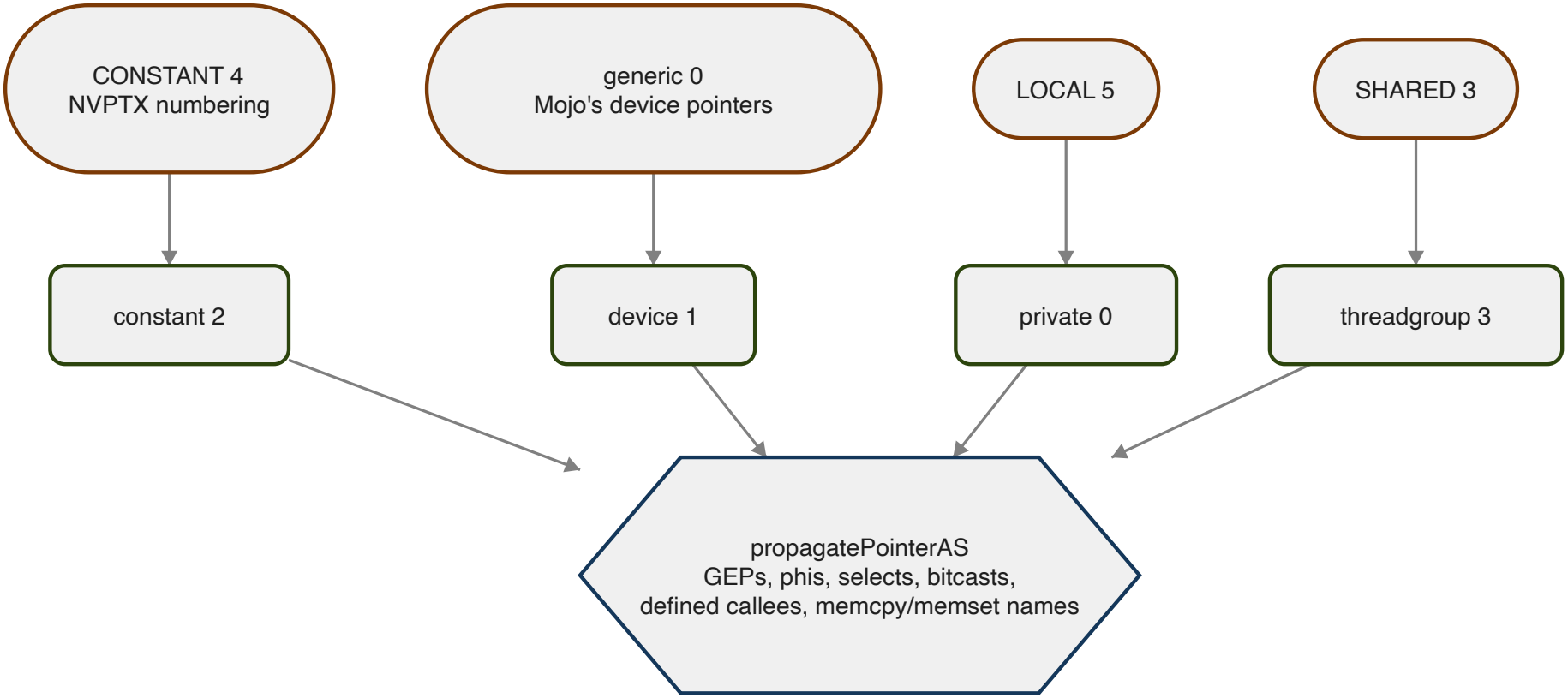
The naming was golden-sampled from a kernel casting scalars and vectors both ways:

```
air.convert.f.f32.s.i32      air.convert.s.i32.f.f32
air.convert.f.f32.u.i32      air.convert.u.i32.f.f32
air.convert.f.v4f32.s.v4i32  air.convert.s.v4i32.f.v4f32
```

that is, `air.convert.<dstKind>.<dstTy>.<srcKind>.<srcTy>` with kind in `{f, s, u}`. Float-to-float is split by width: a scalar `half` or `bfloat` to `float` stays a native `fpext`/`fptrunc`, while the vector form becomes `air.convert.f.v4f32.f.v4bf16` and its three siblings. The vector native form is not rejected either — it is *computed wrongly by the driver with no error*.

6. Address spaces

This is the load-bearing step, and the oracle finding that opens this document's index is about it. Mojo elaborates device pointers in the *generic* address space, because NVPTX accepts that, and numbers the others NVPTX's way. AIR has different numbers and no generic space at all.



`remapAddressSpaces` renumbers module globals and propagates. `propagatePointerAS` follows a changed pointer through its use graph, mutating derived pointer values in place. Every consumer that was ever missed became a separate bug, and the finding tabulates them: a `select` or `phi` whose other arm was left behind; an `icmp`, whose result is `i1` and so is never visited by a walk keyed on pointer results; a constant operand, usually `ptr null`, which has to be *rebuilt* in the new space rather than cast; the overloaded memory intrinsics, which encode address spaces *in the name* and so have to be re-resolved — `refreshOverloadedMemIntrinsics` — rather than retyped; a defined callee whose body still saw the old space; a nested aggregate

whose inner device pointer stayed generic silently; and code arriving *after* the pass, from inlining. *If you write one of these, write all five. They are the same bug and they will surface weeks apart otherwise.*

`deviceizeCapturedPointers` handles the case the fork never had to: a pointer loaded out of the constant argument buffer or pulled out of a capture struct with `extractvalue` is itself a device pointer, and it is retyped to `addrspace(1)` using Apple's own idiom, `inttoptr i64 %x to ptr addrspace(1)`. Not `addrspacecast`: AMD's Metal backend needs the cast to keep pointer provenance for its buffer-resource lookup, and Apple's has no generic space to cast *from*. The port inherited the AMD choice and, in the finding's words, *wrote a grid of zeroes with it. The reasoning behind the original was sound for its target; that is exactly why it survived the port unexamined.*

After the kernels are legalised, `dropNoOpAddrSpaceCasts` removes casts that retyping has made same-space — a same-space `addrspacecast` is invalid IR, which Apple's reader reports only as `Invalid record`. It runs after and not before because cleaning up first misses everything the legalisation is about to create; and they also arrive from the frontend, whose `unsafe_address_space_cast` has nothing requiring source and target to differ.

7. No doubles, and no 128-bit integers

Metal has no 64-bit floats anywhere — MSL has no `double` — and no 128-bit integer type. Every instruction in every function is checked, result and operands, and a kernel that uses either is *refused* with an error that names it:

```
float64/float128 not supported on Metal/AIR (in kernel 'k'): Metal has
neither a double nor a 128-bit integer type
```

— the integer case carries its own wording ("integers wider than 64 bits"). That is deliberately a located compile error rather than a silent demotion. The firewall's `f64` rule, inherited from the proof-of-concept backend and left at Log, records the alternative that backend chose.

8. Each kernel gets an AIR signature

`legalizeKernel` rewrites one exported kernel. Parameter lists are immutable in LLVM, so it builds a fresh function, splices the body in, and returns the new function together with the per-argument metadata list. Three things change.

Builtin shims become trailing parameters. The remaining `air.*` calls that are not runtime functions — thread and threadgroup positions, sizes, the simdgroup indices — are collected per kind; the new signature is the original parameters plus one parameter per builtin kind used. A dimension suffix in the original name (`.x`, `.y`, `.z`) becomes an element extraction from the three-wide parameter. Each such parameter carries its metadata tag: `air.thread_position_in_grid`, `air.thread_position_in_threadgroup`, `air.threadgroup_position_in_grid`, `air.threads_per_grid`, `air.threads_per_threadgroup`, `air.threads_per_simdgroup`, `air.thread_index_in_threadgroup`, `air.thread_index_in_simdgroup`, `air.simdgroup_index_in_threadgroup`, `air.threadgroups_per_grid`.

Pointer parameters move to the device address space, because Mojo elaborated them generic — *this rewrite is the address-space half of the closed MetalAIRPass*. By-value aggregate parameters — the capture blob a closure kernel carries — become a `constant T&` in `addrspace(2)`, used where it stands; by-value scalars become an `addrspace(2)` parameter loaded at entry. Both remember the original type, so the field layout stays recoverable. Dynamically-sized threadgroup globals are rewritten into `addrspace(3)` parameters the launch sizes through `setThreadgroupMemoryLength`.

Captured pointers are not hoisted. The x86-64 fork this descends from pulled every device pointer inside a capture struct out into its own kernel buffer parameter, because AMD's Metal backend cannot resolve a raw address to a buffer resource. On Apple silicon that is *actively harmful*: thirty-one device buffers is the limit and hoisting spends one per captured pointer, so a kernel Apple would bind as a single constant buffer can exhaust it. The canonical Apple shape, verified with a golden sample of a kernel taking `constant Caps& { device float* p; device float* q; uint n; }`, keeps the pointer in the struct and describes it with nested metadata on *one* buffer:

```
!12 = !{i32 0, !"air.indirect_buffer", !"air.buffer_size", i32 24,
        !"air.location_index", i32 0, i32 1, !"air.read",
        !"air.address_space", i32 2, !"air.struct_type_info", !13, ...}
!13 = !{i32 0,  i32 8, i32 0, !"float", !"p", !"air.indirect_argument", !14,
        i32 8,  i32 8, i32 0, !"float", !"q", !"air.indirect_argument", !15,
        i32 16, i32 4, i32 0, !"uint",  !"n", !"air.indirect_argument", !16}
!14 = !{i32 0, !"air.buffer", !"air.location_index", i32 0, i32 1, ...
        !"air.address_space", i32 1, ...}
!16 = !{i32 2, !"air.indirect_constant", !"air.location_index", i32 2, ...}
```

Two details are easy to miss and are written down beside the code: the `struct_type_info` is a *flat* tuple per field — offset, size, zero, type name, field name, `air.indirect_argument`, node — and the nested `location_index` is its own namespace, not the top-level buffer numbering. Emitting this is the open item tagged `TODO(air-indirect)`. It is not a correctness blocker — `deviceizeCapturedPointers` retypes the loads and the runtime keeps the

pointees resident — but it is what would let residency be narrowed from *every live allocation on the encoder* to the reachable ones. Chapter 5 returns to that cost.

The kernel entry in `!air.kernel`

Every legalised kernel is registered in the module's `!air.kernel` named metadata: a node naming the function, an empty node where Apple's front end puts kernel-level properties, and the list of per-argument nodes. The per-argument shape for an ordinary device buffer and a thread position, in the form the golden samples show, is:

```
!air.kernel = !{!0}
!0 = !{ptr @k, !1, !2}
!1 = !{}
!2 = !{!3, !4}
!3 = !{i32 0, !"air.buffer", !"air.location_index", i32 0, i32 1,
       !"air.read_write", !"air.address_space", i32 1,
       !"air.arg_type_size", i32 4, !"air.arg_type_align_size", i32 4,
       !"air.arg_type_name", !"float", !"air.arg_name", !"out"}
!4 = !{i32 1, !"air.thread_position_in_grid",
       !"air.arg_type_name", !"uint", !"air.arg_name", !"tid"}
```

The vocabulary the backend writes — `air.buffer`, `air.location_index`, `air.address_space`, `air.read` and `air.read_write`, `air.arg_type_size`, `air.arg_type_align_size`, `air.arg_type_name`, `air.arg_name`, `air.indirect_buffer`, `air.indirect_argument`, `air.indirect_constant`, `air.struct_type_info`, `air.buffer_size` — is exactly the set that appears in Apple's output for the same kernel, and the reflection the runtime asks for in chapter 5 is Metal reading this metadata back.

After the kernel loop, and not before it, `rebuildMismatchedSignatures` reconciles any function whose parameters were retyped but whose `FunctionType` was left behind — the ordering is stated in the source because `legalizeKernel` is what creates the mismatch.

9. Module flags and versions

The six `air.max_*` limits from the profile go in as module flags — beside two stock clang flags, `wchar_size` and `frame-pointer`, that the golden samples also carry — then the identification metadata, *per the golden sample* and, the comment notes, *NOT guessed from* the feature string:

```
!air.version           = 2, 8, 0
!air.language_version  = "Metal", 4, 0, 0
!air.compile_options   = "air.compile.denorms_disable", ...
!air.source_file_name  = "mojo-kernel"
"SDK Version"          = 26, 0
```

10. Scrub

The last step removes what must not reach the reader: the host's `target-cpu`, `target-features` and `tune-cpu` attributes from every function (the module is handed to a clang-17-era `metal -x ir` as *text*, which is stricter than the bitcode reader about attributes it does not know); the argument attributes that era has no record for (`byval`, `captures`, `range` and friends) and `dso_local` on non-local functions; the exported-kernel passthrough attribute the MLIR lowering stamped; every `!llvm.loop` metadata node, unless `APPLEGPU_AIR_KEEP_LOOP_MD=1` asks for a comparison; and finally the `air.apple_arch` node, which was for legalisation, not for the artefact.

The loop metadata line has a story. Before the unroller existed it was an experiment knob; when the unroller's gate began marking loops it had declined with `llvm.loop.unroll.disable`, those marks leaked through to Apple's compiler, which honoured them, and the rolled matmul dropped to 116 GFLOP/s. Chapter 6 has the table. Loop metadata does not go to the driver.

4. What went wrong

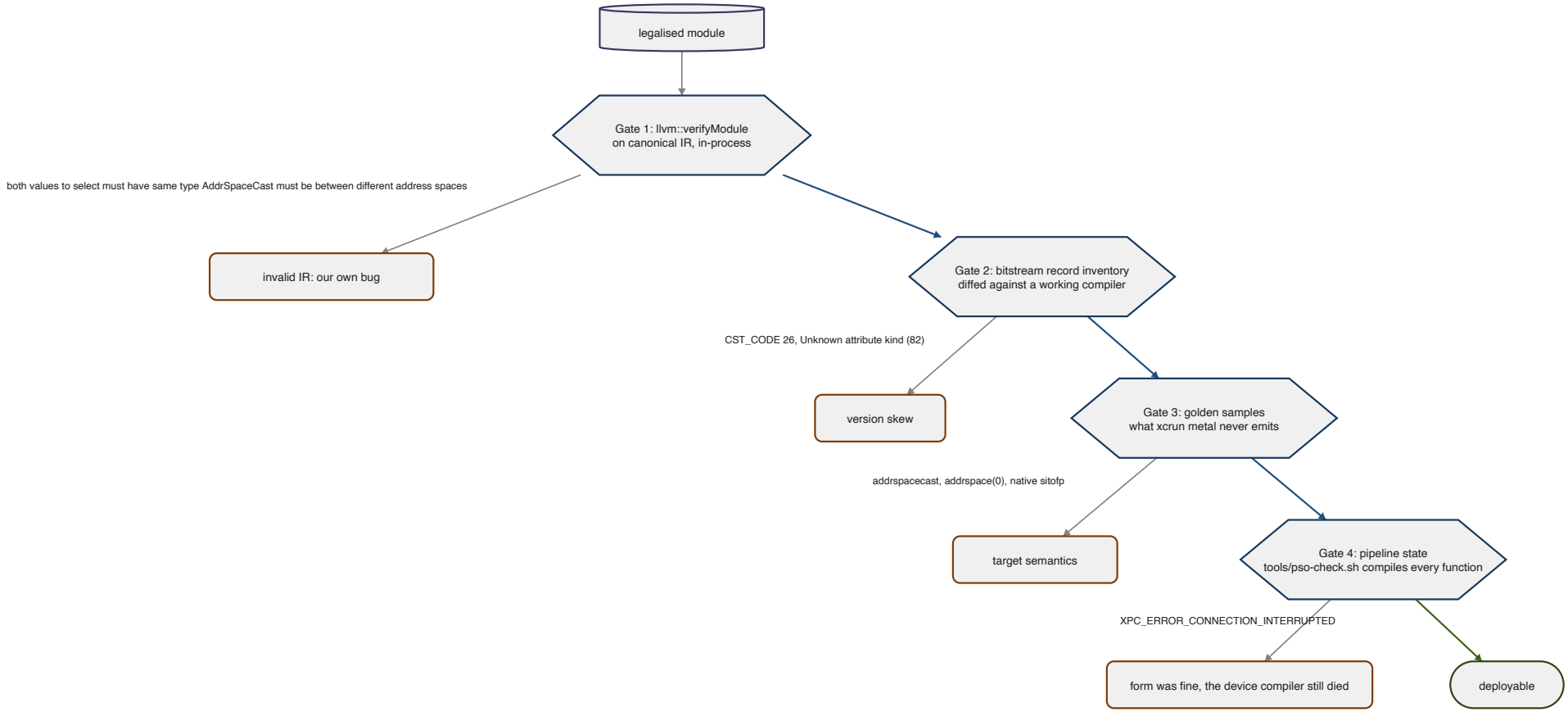
The defects in this chapter are arranged by the gate that would have caught them, because that is the shape the work took. For the first weeks every failure started from zero: three unrelated causes produced the same error message, and until they were separated there was no way to know which kind of wrong a module was. The diagnostics finding calls the gates *the single highest-leverage thing in this repository — adding these turned defects that cost hours each into defects that cost one cycle each*.

Three causes, one message

What the reader says	What it can mean
`Invalid record`	invalid IR that no reader accepts; **or** a modern construct the frozen reader predates; **or** a same-space `addrspacecast`
`Unexpected bitcode file!`	`metallib`'s answer to everything, including a `poison` constant and a call whose explicit type disagrees with its callee
`XPC_ERROR_CONNECTION_INTERRUPTED`	the compiler *service* died at pipeline creation: a misspelled symbol, a dead declaration, a native cast, an `i4`, a vector `llvm.fma` — none named

> *No function. No instruction. No hint that a type might be involved. Three different defects produced this identical message, and nothing distinguished them until the module was compiled for the device in isolation.*

The three causes are: **invalid IR** — the backend's own bug, rejected by modern LLVM too; **version skew** — valid modern IR the reader predates; and **target semantics** — not skew, a different machine model. Each gate isolates one.



Gate 1: verify before serialising

`llvm::verifyModule`, immediately after legalisation and before the downgrade. Chapter 2 quoted why the placement matters. What it caught was humbling: a good share of what the reader had been rejecting was not AIR-specific at all.

> *select with mismatched pointer types and a same-address-space addrspacecast both reached metallib and came back as "Invalid record"; run through the verifier they are "both values to select must have same type", which says where to look.*

The finding adds a warning that proved true: *expect it to reject things that currently "work"*. Two long-standing invalid-IR defects had been shipping quietly because the reader tolerated them. *Fix the IR, not the gate.*

The address-space defect class

The largest family Gate 1 exposed was the one chapter 3 described from the mechanism side: a retyped pointer whose consumer was left behind.

Consumer	Symptom	Fix
`select` / `phi`	`both values to select must have same type`	reconcile the *other* arm
`icmp`	`Both operands to ICmp are not of the same type!`	result is `i1`, so a walk keyed on pointer results never visits it
constant operands	the same, usually `ptr null`	rebuild the constant in the new space
overloaded intrinsics	`Call parameter type does not match function signature!`	`llvm.memcpy` encodes the spaces in its *name* — re-resolve
defined callees	body still sees the old space	retype the parameter and recurse
nested aggregates	a device pointer stays generic, silently	extract the inner struct first
code arriving after the pass	the same, for one callee's pointers	re-run after inlining

And the one the verifier cannot see, because the IR is well-formed: a device pointer left generic. Its signature is an output buffer of zeroes with no error anywhere. That has *two* causes on Apple, needing opposite fixes, and one run separates them:

`MTL_SHADER_VALIDATION=0`	`=1`	Cause
passes	fails	non-residency: the buffer is not `useResourced`
fails	fails	a device pointer left in `addrspace(0)`

The port guessed residency first, on the strength of the earlier finding, and was wrong; the loads and stores of one kernel had disagreed, twelve generic against forty-two device, and *only half the kernel was wrong*. The one-line census that settles it — count the address space on every `load` — is in the finding, and the rule it produced is absolute: *any ptr without an addrspace in a finished AIR kernel is a defect unless it is alloca-derived, and if the module has no alloca at all, there is no unless.*

Gate 2: the bitstream, not the disassembly

The version-skew table, every row measured:

Construct	Since	Symptom
unary `fneg`	LLVM 8	writer hard-crash
`freeze`	LLVM 10	writer hard-crash
`poison`	LLVM 12	`CST_CODE 26` → `Unexpected bitcode file!`
`llvm.stepvector`	LLVM 12	XPC crash at pipeline creation
attribute codes ≥77 (`range`, `nofpclass`, `allockind`)	various	`Unknown attribute kind (82)` / `Invalid record`
GEP no-wrap flags (`nusw`, `nuw`)	LLVM 19	text form rejected; encoded flags break the driver
fast-math flags on FP *casts*	modern	one flagged cast kills the module
`memory(none)`	LLVM 16	**not** a rejection cause here — see below

The last row is a correction, and the story behind it is the most expensive tooling lesson in the port. Disassembling a rejected `.air` with `llvm-dis` showed `memory(none)` on `llvm.umax.i64` — a known-bad LLVM 16 attribute, an open-and-shut diagnosis. Several rounds of "strip it harder" changed nothing, *because there was nothing to strip*:

> *llvm-dis re-populates intrinsic declarations with their default attributes on load. The attributes it prints are therefore not necessarily the attributes in the file.*

Two defences came out of it. Dump the module from *inside* the compiler, immediately before encoding — that text is what was actually written, and the `.pre.ll`/`.post.ll` artefacts exist for this. And trust the bitstream: `llvm-bcanalyzer --dump` reports records as encoded, and attributes re-derived on load do not appear.

The typed-pointer rule

The real defect in that module was structural, and it is the one rule about typed pointers worth memorising:

> *A call's explicit type must equal the pointee type of its callee operand.*

Under opaque pointers that is vacuous, so Gate 1 is silent. It becomes load-bearing the moment `PointerRewriter` retypes pointers for the old writer, because the rewriter records a retyped callee's new `FunctionType` in its own side table while the call instruction keeps returning its *pre-rewrite* copy. Emit that and the two disagree; Apple's reader says `Explicit call type does not match pointee type of callee operand`, and `metallib` says `Unexpected bitcode file!`.

The writer already had the fix — gated by name to the one intrinsic family it was first hit on. *The rule is general: it holds for any callee the rewriter retyped, which is any callee taking a pointer once a kernel's address spaces are legalised.* Ungating it took the gather/scatter/index cluster from sixteen kernels rejected to sixteen accepted. *Worth checking whether your own port has the same shape — a correct fix wearing a name check that hides how general it is.*

Gate 3: what the vendor never emits

Chapter 1 gave the list. Each entry was a separate day: `addrspacecast` where Apple's idiom is `ptrtoint`/`inttoptr`; `addrspace(0)` on anything but an alloca; and the four native integer-float casts, which `metallib` accepts and the device compiler dies on. The one tool that names a defect is worth repeating, because it should be the *first* thing run against a rejected module, not the last:

```
$(xcrun -f air-opt) kernel.air -o /dev/null
```

`air-opt` is stricter than `metallib` — it rejects bitcode that `metallib` accepts and that runs correctly, including Modular's own — so it is a diagnostic, never a verdict.

Gate 4: compile it for the device

All three gates above check *form*. None compiles the module for the GPU, and three defects passed all three and still killed the compiler service:

- **Dead intrinsic declarations.** The downgrade folded every `llvm.stepvector` call and left the `declare` behind. The reader resolves every declared symbol, so the module still died — and nothing in the IR referred to it. Hence the sweep in chapter 3.
- **Vector fma must be air.fma.<ty>.**
- **bitcast <4 x i1> to i4.** *Any integer type that is not 8/16/32/64 bits is a defect in a GPU module, and it will arrive from the optimiser rather than from your own lowering.* Although — the firewall's `odd-int-width` rule is *Log*, not *Fail*, because `test_grid_dim` emits a `trunc i64 to i2` and passes. Generalising from the `i4` case was wrong; the reader tolerates some odd widths, and the rule narrows only with evidence.

Only Gate 1 runs in-process. Gates 2 and 3 live in `spikes/air-gates.sh` in this repository, and Gate 4 is `tools/pso-check.sh` in the oracles repository: it loads a metallib, or packages a `.air` first, compiles every function in it to a pipeline state, and exits non-zero if any fails. About twenty lines of Objective-C. Verified in both directions on the artefact that found it — exit 1 before the fixes, exit 0 after — and meant for CI.

The other technique from that finding found two of the three before any bisection: list the declared symbols and compare them with the vendor's for equivalent source. *A symbol the vendor never emits is a defect; a symbol it emits under a different name is a defect. Both are invisible in the instruction stream and obvious in the symbol list.*

The legality firewall

What the gates learned is written into `AirLegality.cpp` as a rule table. Each rule has an action — `Fail`, `Log`, or `Permit` — and an *evidence* field that says how the backend knows: `measured` means this backend shipped the defect and the fix was verified on an M4; `air-poc` means it comes from the out-of-tree LLVM AIR backend and stays at Log until confirmed here; `semantic` means it is a property of the machine, not of a reader.

Rule	Action	Evidence	What it catches

`mask-bitcast`	Fail	measured	`<N x i1>` ↔ `iN` — the form measured to pass verifyModule, metallib and air-opt, then kill the service
`three-way-compare`	Fail	measured	a surviving `llvm.scmp/ucmp`; the expected count is zero, so there is no false positive to find
`native-int-float-cast`	Fail	measured	`sitofp/uitofp/fptosi/fptoui` — must be `air.convert.*`
`vector-fp-cast`	Fail	measured	vector `fpext/fptrunc` — computed wrongly with no error
`vector-llvm-fma`	Fail	measured	vector `llvm.fma` — must be `air.fma.<ty>`
`unknown-air-symbol`	Fail	measured	an `air.*` declaration whose name or type is not in the contract
`divergent-barrier`	Fail	semantic	`air.wg.barrier` control-dependent on a thread, lane or simdgroup identity
`odd-int-width`	Log	unproven	integer widths outside 1/8/16/32/64, other than the mask case
`unresolved-external`	Log	measured	a declaration-only symbol the reader may not resolve
`generic-deref`	Log	measured	a load/store/atomic through a non-alloca `addrspace(0)` pointer
`addrspacecast`	Log	measured	Apple emits none; a same-space cast is invalid outright
`dead-intrinsic-decl`	Log	measured	an `llvm.*` declaration nothing calls
`unmapped-llvm-intrinsic`	Log	unproven	a vector `llvm.*` math intrinsic with an `air.*` equivalent — *probably still too broad*
`i64-simd-shuffle`, `f64`, `int-to-bf16`, `nonvolatile-loop-load`, `unguarded-scalar-store`	Log	air-poc	inherited from the proof-of-concept backend, unconfirmed here
`nan-minmax-unwrapped`	Permit	air-poc	kept only as documentation; it cannot work as a detection rule

Two rules are worth reading for the reasoning alone. `unmapped-llvm-intrinsic` says of itself that *the evidence is weaker than it looks*: `llvm.fma.v4f32` was measured to kill pipeline creation, but `llvm.maxnum.v4f32` is emitted 661 times across ten tests and evidently tolerated — *treat a hit here as a question, not an answer, until the specific intrinsic has been tested*. And `nan-minmax-unwrapped` is switched off because it *cannot* work:

AIR's `fmin`/`fmax` drop NaN, which is correct for `llvm.minnum`/`maxnum` and wrong only for `llvm.minimum`/`maximum`, and after renaming the two are indistinguishable — Apple's own output trips it four times. The correctness belongs in the `guard-nan-minmax` transform, which wraps only calls it renamed.

Any rule can be downgraded for a comparison — `APPLEGPU_AIR_RULES=rule=log` — but the table is the shipping posture, and a Fail that has to be turned off is itself a finding waiting to be written.

The `divergent-barrier` rule is the one piece of real analysis in the file. It treats the kernel arguments tagged by `!air.kernel` as the authoritative thread-identity sources, follows their SSA dependencies into conditional branches, and uses dominator and post-dominator analysis to tell a lane-dependent branch that *reconverges* before one shared barrier (legal) from a barrier selected, skipped or iterated by a thread-dependent condition (fails). Threadgroup position and launch-size arguments are uniform at workgroup scope and do not taint a branch. It found three latent divergent barriers in the basics kernels (D8b) that had been passing on hardware by luck.

The defects that were not about IR at all

Ledger	What happened	What it taught
D21	the feature-warning fix stripped `target-cpu` before legalisation read the arch from it; every GPU program failed with `no Apple AIR target profile for arch ''`, and the change was pushed as verified	the check had run a binary the *previous* build left behind; a counter that cannot tell "zero" from "never ran" is not a check
D22	six knob experiments in a row measured the same number and emitted identical AIR — none had run; `~/.cache/modular/.mojo_cache` keys on source and compiler version string, not environment, not a local rebuild	a null experiment is a cache hit until proven otherwise; every knob prints `[air-knobs]`, and the suites run with a fresh `MODULAR_CACHE_DIR`
the Bazel `PATH`	`xcrun` was not found because the compile action has no `PATH` at all	the bitcode is generated first; a packaging failure reads like a codegen failure

The ledger, continued:

Ledger	What happened	What it taught

the gate marks	`llvm.loop.unroll.disable` on loops the gate declined leaked to Apple's compiler, which honoured it; rolled matmul fell to 116 GFLOP/s	nothing the device pipeline says to itself may reach the artefact
D6	one rowwise subkernel variant kills the compiler service; reduction from retained artefacts still open	keep `.air` before packaging, name artefacts per kernel, include the profile
D23	`DeviceExternalFunction` launches segfaulted because the sizes never crossed the ABI; now a clean contract error	a null sizes pointer is the Apple argument-view discriminator, not "no sizes" — the runtime must be told which protocol the caller speaks

D21 has a memory attached to it in this project's notes, and the rule it left is short: a compiler change is verified by `check-examples.sh` and `check-gamepane.sh`, never by the one program that motivated it, and never on a bench number alone.

5. The runtime

Everything so far produces bytes. `AppleGPURT` is what turns them into a dispatch, and it has its own failure surface — one the oracle findings call *a second, unshared failure surface*, because everything else in the port is about what the compiler emits and this is about the boundary underneath it.

Implementing the other half of a declared interface

Mojo's `DeviceContext` is a thin Mojo wrapper over a C ABI of `AsyncRT_*` functions: `AsyncRT_DeviceContext_create`, `_createBuffer_async`, `_HtoD_async`, `_DtoH_async`, `_compileFunction`, the launch entry points, the `AsyncValue` retain and release family, streams, sub-buffers, and so on.

The bindings in `device_context.mojo` declare every symbol, so the *interface* is fully specified in ordinary open source — the names, the argument types and the return conventions are all simply there to read. What that file does not carry is a Metal implementation behind those declarations, nor prose describing the intended semantics of several of the calls, and, as the finding says, *that is where the cost is*: the shape of each function is given, and what it is supposed to *do* had to be settled by writing it and testing the result. `AppleGPURT.cpp` and `AppleGPUMetal.cpp` are that implementation for Metal on Apple silicon.

It is written in plain C++ over the Objective-C runtime — `objc_msgSend` casts, the metal-cpp technique — so no Objective-C++ toolchain support is needed anywhere in the build. One consequence had to be learned: Metal hands back *autoreleased* objects from `commandBuffer`, `computeCommandEncoder` and the blit encoder, and a plain C++ file gets no `@autoreleasepool`. On a thread that has one — AppKit's main thread — they drain at the end of each event-loop turn and nobody notices. On a worker thread that does not, *they are never released at all*. The runtime now scopes its own pool around every hot-path call that allocates one; a few cold paths, like context creation, still leak their strings.

Unified memory changes the shape of everything

The header comment sets the design in one paragraph:

> *On Apple Silicon the CPU and GPU share one pool, so every buffer is storageModeShared and both HtoD and DtoH are a plain memcpy — no staging buffer, no blit encoder, no command buffer. The x86-64 fork could not do this: a discrete Vega II has its own HBM2, so device buffers had to be storageModePrivate and every transfer went through a staging blit.*

So `isHost` no longer means "host-visible" — everything is. It means only "the address handed to Mojo is a CPU pointer", which is still a real distinction. The staging paths are kept behind a `hostVisible` flag so a future `storageModePrivate` mode — worthwhile for GPU-only buffers, which Apple can keep in a compressed layout — can switch it back per buffer.

The address registry

Device pointers handed to Mojo are `MTLBuffer` `gpuAddress` values. A global interval map resolves any device address back to its owning buffer and offset. That is how a kernel launch binds a pointer argument — `setBuffer` with a resolved offset — *without trusting struct layouts we don't own*, and it is how a sub-buffer, a pointer into the middle of an allocation, or a device address a kernel captured by value can all be bound the same way.

The registry has a mirror: an `MTLResidencySet` holding every root buffer. The set is attached to the command queue once; membership is edited when an allocation is created or destroyed and committed then, so the driver keeps everything in it resident for every command buffer on that queue with no per-dispatch work at all. It is per *device*, not per context, because a capture blob may carry any live device address, so two contexts on one device share a set attached to both queues. The section on residency below says what this replaced.

Reflection is the argument contract

Loading a kernel is Metal's own three steps — `newLibraryWithData`, `newFunctionWithName`, `newComputePipelineStateWithFunction:options:reflection:` — with the reflection option set deliberately:

> *Without the argument contract the launch path has to guess whether an 8-byte value is a device address to bind or scalar bytes to copy, and a guess is wrong in one direction or the other.*

What comes back is Metal's reading of the `!air.kernel` metadata chapter 3 emitted: for each buffer index, whether the slot is a buffer or a constant, its address space, its size. The runtime keeps that as `argSlots`, indexed by buffer index, and the launch path binds against it rather than against anything the caller says. `STATUS.md` records the moment that became true: *reflection is now authoritative for compiler-generated kernels*.

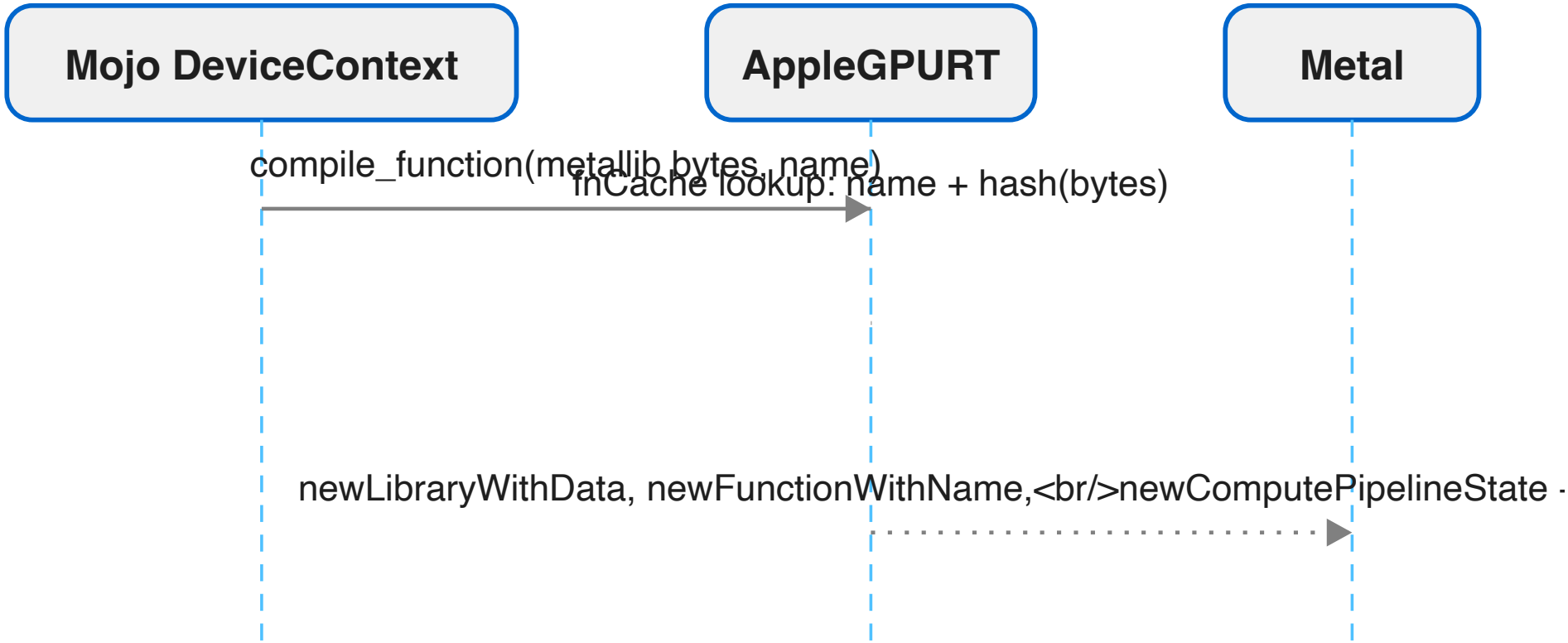
One ambiguity is documented beside the slot table and is worth knowing about. The port's kernels declare device parameters with an opaque pointee, so Metal reports the buffer's data type as *none*, and that cleanly identifies a device buffer. An MSL kernel declares `device float*`, so Metal reports `Float`/4, and the same test would call it a constant. *Same reflection API, opposite meaning, and no way to tell which you are looking at without knowing*

where the function came from. So each loaded function carries a flag saying whether the compiler generated it, and reflection is trusted as the contract only when it did.

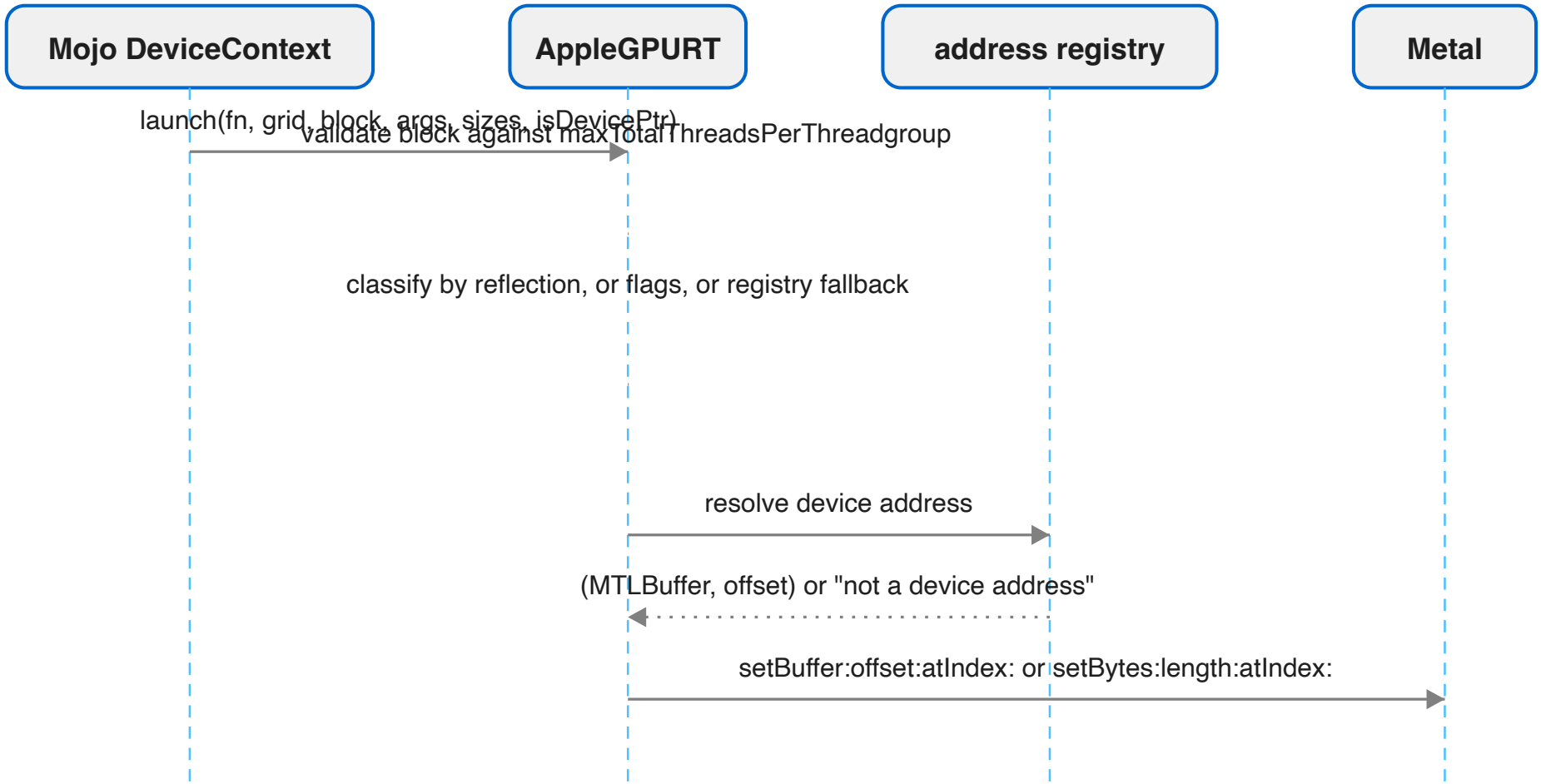
The function cache

Compiling a function used to happen on every call: a fresh `MTLLibrary`, `MTLFunction` and pipeline state, every time `compile_function` ran — and the `enqueue_function[kernel]` template form every example uses runs it per launch. The bytes arrive in a fresh `String` each time, so their address is no identity. A per-context cache keyed by function name, the module length, a 64-bit hash of the bytes, and the kernel's maximum dynamic threadgroup bytes brought that from 17.8 µs per call to 0.8–1.2 µs. It is the first of the three dispatch fixes in chapter 6.

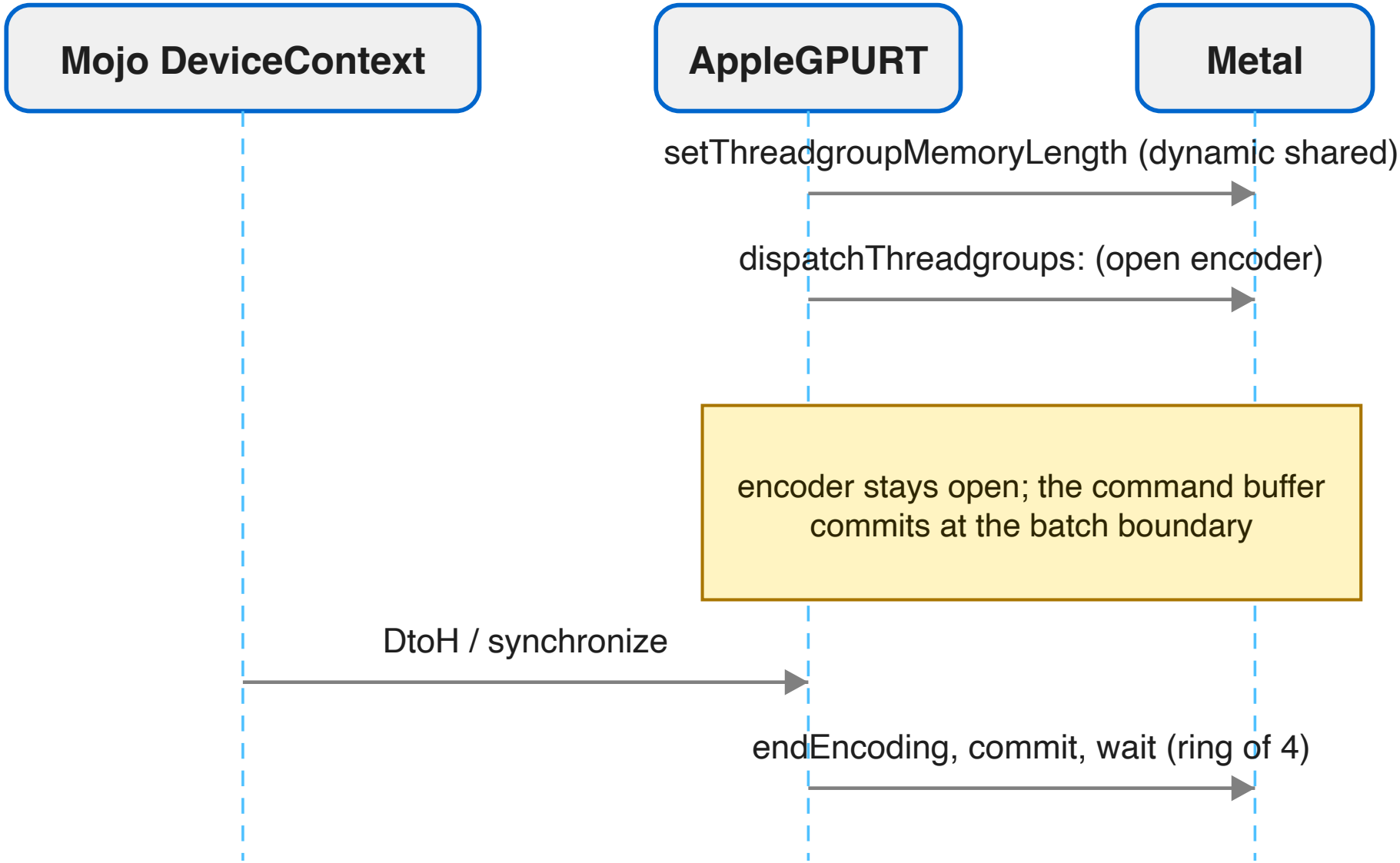
A launch, step by step



The launch path is where the argument contract is actually resolved, and it is the half that runs on every dispatch rather than once:



Once every argument is bound, the dispatch itself is three calls — and the encoder deliberately stays open behind them:


Mojo DeviceContext
AppleGPURT
Metal
setThreadgroupMemoryLength (dynamic shared)
dispatchThreadgroups: (open encoder)
encoder stays open; the command buffer commits at the batch boundary
DtoH / synchronize
endEncoding, commit, wait (ring of 4)

The argument model is stated at the top of the launch function:

> *Per-arg value pointers + sizes + an is-device-pointer flag per argument. Pointer args hold a 64-bit device address; we resolve it to (MTLBuffer, offset) and bind with setBuffer — which also makes the resource resident. Scalar args are bound with setBytes. Argument index == buffer slot index.*

The flags are wrong for captures, so the contract decides

The Mojo side marks every capture slot as "not a device pointer" — *captures are raw values, never device buffers* — which holds on CUDA, where a captured pointer is just an integer the kernel dereferences. It does not hold here. A capture that the backend has typed as a device buffer parameter expects a *binding*, not the address bytes. Binding one with `setBytes` put the address value where the kernel expected the buffer base, and the kernel wrote 1.0 through it into nothing; the test that caught it, `test_static_layout_capture_argcount`, failed with its output buffer still holding the –1.0 fill.

The first fix resolved every pointer-sized argument against the registry regardless of the caller's flag — *this runs even when the caller supplied explicit flags, and must* — and that heuristic is what caught the bug. It is also a guess that can in principle misfire on a scalar whose value happens to land inside a live allocation, so it did not stay the answer. The principled fix is the one the comment names: *ask the pipeline what each argument is*. For a compiler-generated kernel, reflection **is** the contract — every slot must be described, a caller flag that disagrees with it prints a line and loses, and the registry heuristic survives only as the fallback when there is no reflection at all. Resolving a device address to `(MTLBuffer, offset)` through the registry is still how every such argument is *bound*.

Metal can silently no-op a dispatch

> *Metal can silently no-op a dispatch whose threadgroup is too large for the pipeline (SDL #15241); validate against the pipeline's own limit and fail loudly instead.*

`APPLEGPU_TRACE_LAUNCH=1` prints the pipeline's `maxTotalThreadsPerThreadgroup` and `threadExecutionWidth` beside every launch for exactly this class of question.

Residency

`setBuffer` makes the bound resource resident. A pointer a kernel reaches *through* a capture blob — a struct holding a device pointer, passed as one constant buffer — is never bound, so Metal has no idea the pointee is in use. The measured behaviour, on an M4 Max, is the trap chapter 4 tabulated: with shader validation off, the read *passes silently* because a small, recently-written buffer happens to be resident; with validation on, every read returns zero. *So it is not fragile, it is already wrong, and it looks fine.*

The first correct answer was `markAllResident()`: declare every live root buffer to every compute encoder, one `useResource:` call per buffer per dispatch, under the registry lock. Correct, and *O(live allocations) per dispatch* — the cost of a dispatch grows with every unrelated allocation in the process, which the experiment log named as the next runtime scaling defect. The residency set inverts that cost:

> *The set is attached to the command queue once; membership is edited when an allocation is created or destroyed and committed then — O(changes), off the dispatch path entirely. Metal keeps everything in an attached, committed set resident for all command buffers on that queue, which is exactly the guarantee useResource was providing, minus the per-dispatch walk.*

`APPLEGPU_COARSE_RESIDENCY=1` restores the walk as a diagnostic escape hatch (presence-parsed: any value at all, `=0` included, turns it on — the same is true of the `APPLEGPU_TRACE_*` switches, unlike the value-parsed launch-mode switches below). What the set still cannot do is shrink below *all of them*: narrowing residency to the pointees a kernel can actually reach is what the `air.indirect_buffer` metadata in chapter 3 would buy, and that half is still open.

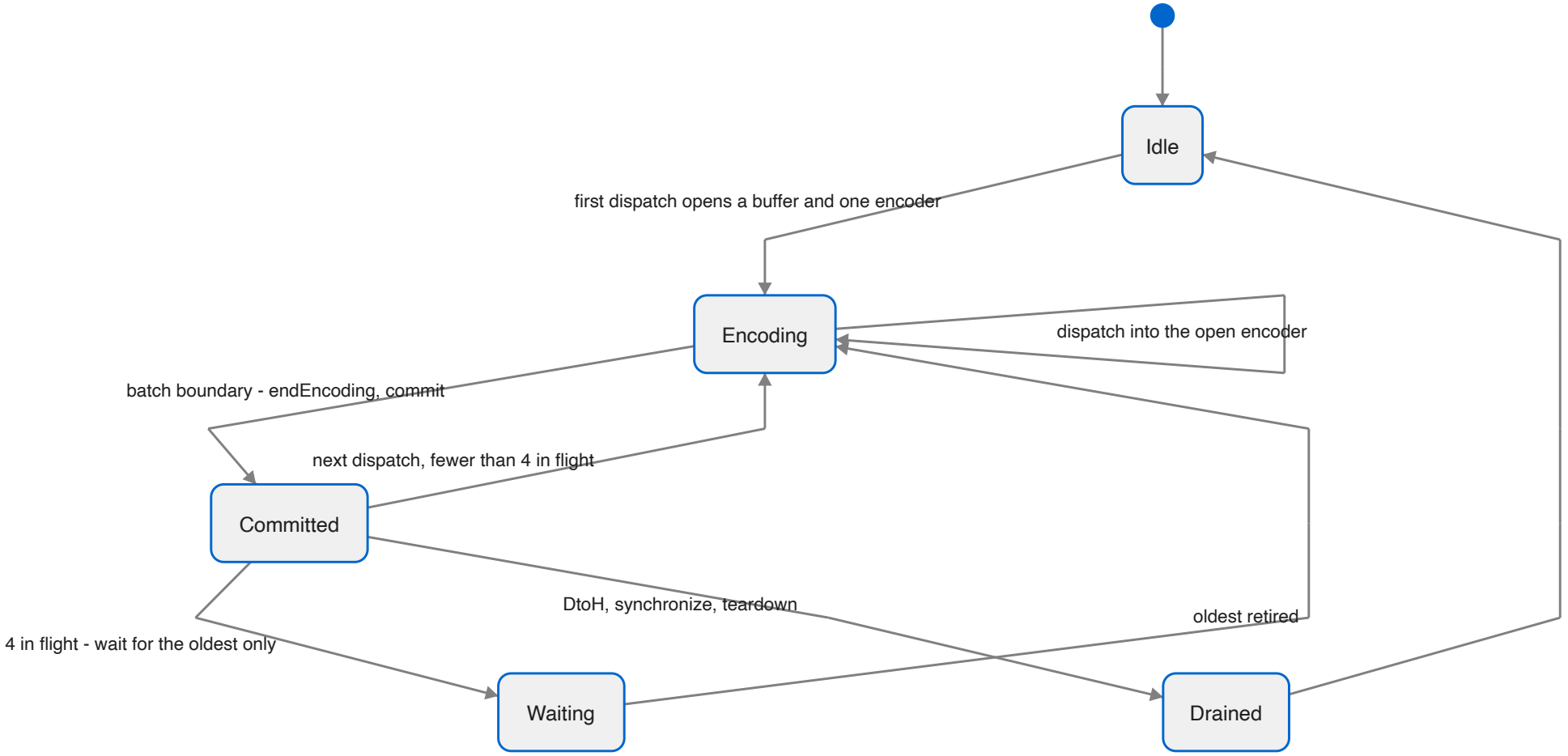
Batching, encoders, and the ring

Launches queue and batch by default and drain at synchronisation or host-observation boundaries — any `DtoH`, any `synchronize`, teardown. Three decisions set the cost of a dispatch, and all three were changed in one session after measuring Metal's own floor (chapter 6):

1. **One compute encoder per batch**, not one per dispatch. An encoder boundary is a GPU-side pipeline drain, and Metal charges 3.5 µs for it against 1.0 µs for a dispatch inside an open encoder. The serial dispatch type orders dispatches within an encoder, so a dependent chain needs no boundary

between links. The bench does not assume this: its ordering probe runs 1,000 and 5,000 dependent `+= 1` dispatches and asserts exactly 1,000 and 5,000, batched and unbatched.

2. **A ring of command buffers** instead of commit-and-drain at every 64th dispatch. The drain blocked the host until the GPU was empty and then idled the GPU while the next batch was encoded; the ring waits for the *oldest* batch only, and only when four are outstanding.
3. **The function cache** above.



Every read path drains before it copies, state is locked, and three switches exist for isolating a problem: `APPLEGPU_SYNC_LAUNCH=1` restores the synchronous bring-up mode, a round trip per dispatch; `APPLEGPU_BATCH_DISPATCHES=0` keeps the queue but gives each dispatch its own command buffer; and the older `APPLEGPU_ASYNC_LAUNCH=0` disables asynchronous launch without claiming the synchronous mode — `SYNC_LAUNCH` wins when both are set. Failures a read path cannot report (a host pointer copy, a buffer destroy) are parked and surface at the next `synchronize`.

The ABI has its own defect class

Two of the findings under this heading are not about Metal at all. They are about what an `Optional[Pointer]` is at a C boundary, and they bit from opposite directions weeks apart.

`Optional[Pointer[T]]` is niche-optimised: `Pointer` is non-nullable, so the optional is pointer-sized with null standing for `None`. Both bugs come from believing that means it *is* a pointer. In one direction, `arg_sizes` — declared `OptionalPointer` in `DeviceContext.enqueue` — arrived at the C side as NULL, always, because the value is a nested memory-only aggregate the C ABI lowering does not unwrap; the callee fell back to pointer-sized argument sizes, right for buffers and wrong for scalars. In the other, a zero-byte buffer reporting device address 0 reappeared in Mojo as an *empty* optional and aborted. The rule the finding draws: *at an external_call boundary, treat Optional[T] as an aggregate, not as a nullable scalar, in both directions. If NULL is meaningful, spell the parameter as an integer address and convert at the edge.*

That is also the shape of D23. `DeviceExternalFunction` — the path that launches a kernel supplied as bytes rather than compiled from a Mojo `fn` — never passed argument sizes, and a null sizes *pointer* on this runtime is not "no sizes": it is the protocol discriminator meaning *the first entry is the Apple argument view*, so the runtime reinterpreted the bare pointer array as that struct and segfaulted. The enqueuer now always builds and passes a real sizes array, and what remains is routing that path through the same checked argument view the compiled-kernel path uses; a size that disagrees with the kernel's declared constant bytes is now a clean contract error rather than a crash.

6. Making it fast

"Reasonably fast" has a definition in this port: within a few percent of what Modular's released compiler and Apple's own compiler produce for the same kernel on the same machine, with dispatch overhead at Metal's own floor. Both halves were measured before they were true, and neither came from adding an optimisation. This chapter is the four things that mattered, with the numbers that decided each.

Where the port stands	This port	Release 1.0.0	Apple, from MSL	Bench
register matmul, `comptime for` ×16, 2048³, M4	**970** GFLOP/s	956	951	`matmul_reg_unrolled_bench`
register matmul, rolled K-step, M4	958	—	—	`matmul_reg_bench`
FMA chains 1 / 4 / 16 / 64, M4	2,815 / 3,544 / **3,602** / 3,332 GFLOP/s	363 / 1,587 / 2,876 / 3,101	677 / 2,157 / 2,957 / 3,220	`fma_peak_bench`
STREAM triad, M4	**96.9** GB/s of ~120	—	—	`stream_bench`
dispatch, chain of 1,024, precompiled	**0.9–1.0** µs	—	Metal floor 1.0	`launch_bench`
register-blocked matmul 2048³, M4 Max 32-core	**3,056** GFLOP/s (naive kernel: 1,187)	—	—	`bench/README` scoreboard

1. Dispatch cost is an encoder boundary

The first question was not how fast a kernel runs but how much a dispatch costs, because the examples issue dozens of *dependent* dispatches per frame — Fluid needs about thirty-five — and the ledger had it open as D9. The measurement was taken two ways on the same day: `metal-launch.m`, Metal with no runtime in the way, and `launch_bench.mojo`, AppleGPURT through `DeviceContext`. Both run an empty kernel in a dependent chain of *n* dispatches into one buffer, then one commit-and-wait; per-dispatch cost is the chain's wall time over *n*. Both benches live in the oracles repository's `bench/` directory, alongside the findings every number in this chapter is quoted from.

Per dispatch, µs	chain 1	chain 8	chain 64	chain 1024

Metal floor, one encoder per dispatch	154	20.0	4.9	3.5
Metal floor, one encoder for the chain	145	21.7	3.9	**1.0**
AppleGPURT before, kernel precompiled	173	23.6	4.4	4.2
AppleGPURT before, `enqueue_function[k]` per call	184	29.1	8.8	8.6
AppleGPURT before, batching off	173	36.3	21.9	21.7
AppleGPURT before, synchronous	173	156	149	155
AppleGPURT after, precompiled	176	23.7	4.3	**0.9–1.0**
AppleGPURT after, enqueue_function[k] per call	177	25.2	5.1	**1.6**

Three things the columns say. **Chain 1 is the round trip**: about 150 µs for a commit and a wait, whoever wrote the runtime, and a host-observed result cannot cost less than that on this machine. **Chains 8 and 64 are round-trip bound** and barely move. **Chain 1024 is the dispatch itself**: Metal charges 3.5 µs for a fresh compute encoder per dispatch and 1.0 µs inside one encoder, because *an encoder boundary is a GPU-side pipeline drain*. The runtime was at 4.2 — one encoder per dispatch plus its own share — and is now at Metal's one-encoder floor. The three changes were the previous chapter's function cache, encoder reuse, and the ring; `compile_function` went from 17.8 µs to 0.8–1.2 µs per call.

The side effect was the more useful number. STREAM's copy/scale/add/triad medians went from 77 / 84 / 86 / 81 GB/s to 97.6 / 96.8 / 97.8 / 96.9 GB/s, still four of four exact — about 80% of the M4's ~120 GB/s. Its kernels are launched back to back, and each launch had been paying an encoder boundary.

2. The optimisation that had to be turned off

D7 was "unrolled register matmul about 9% behind upstream", and it stayed open for a while because the first six experiments against it were null (the cache trap, below). When they were real, the pattern was immediate.

K-step, 2048³ on the M4, GFLOP/s	This port	Release 1.0.0	Apple, from MSL
rolled (a runtime loop)	**942**	899	—

unrolled ×4, loop of 4	**980**	985	—

The same comparison, continued:

K-step, 2048³ on the M4, GFLOP/s	This port	Release 1.0.0	Apple, from MSL
unrolled ×8, loop of 2	961	962	—
unrolled ×16, straight-line	**870**	956	951

(The straight-line kernel reads 870–871 GFLOP/s run to run; the table and the device-pipeline table below quote different runs of the same configuration.)

Only the fully unrolled kernel was slow, and only here. What the port handed Apple for it: 530 instructions where the release hands 2,607; 32 vector `<16 x float>` operations where the release has 512 scalar FMAs; 16 `<4 x float>` loads plus 64 scalar where the release has 128 scalar loads interleaved per K-step. SLP vectorisation had found the straight-line FMAs and packed them sixteen wide — and an Apple lane is scalar. The costume costs insert and extract traffic the scalar form never had, and Apple's compiler ran it at 871 GFLOP/s against 955 for the scalar form.

Device pipeline	GFLOP/s	What the final AIR looks like
shared O3 (the default before)	871	32 `<16 x float>` ops, 16 `<4 x float>` + 64 scalar loads, 78 folded GEPs
shared O3 **minus SLP and VectorCombine**	**955**	512 scalar FMAs, 128 scalar loads, 126 folded GEPs
device O0	952	the pre-pipeline module: 512 scalar, 128 loads, unfolded
device O1	790	—
device O2	763	—
O3 + threadgroup arrays aligned 16	874	unchanged loads

Two rows in that table are the lesson. O0 — *no device-side optimisation at all* — runs within 0.3% of the best. Apple's compiler is the optimiser; what the device pipeline can contribute is small, and what it can subtract is large. So `wantsVectorization()` and `wantsVectorCombine()` answer no on AIR, `APPLEGPU_AIR_VECTORIZE=1` puts SLP back for a comparison, and the port now sits at 970 against the release's 956 and Apple's 951.

The trap that ate the day before this is D22, and it belongs in any list of what it takes to be fast, because a wrong measurement costs more than a slow kernel. The Mojo compile cache at `~/.cache/modular/.mojo_cache` keys a compiled kernel on its source and the compiler's version string. Not the environment; not, evidently, a local rebuild of the compiler. Six experiments — vectorisers off, threadgroup alignment, three optimisation levels, partial unrolling — measured the same number and emitted byte-identical AIR because none of them ran. Moving the cache aside made all of them real at once. Every experiment knob now prints an `[air-knobs]` line, the check scripts export a fresh `MODULAR_CACHE_DIR`, and the rule is: *no line, no result*.

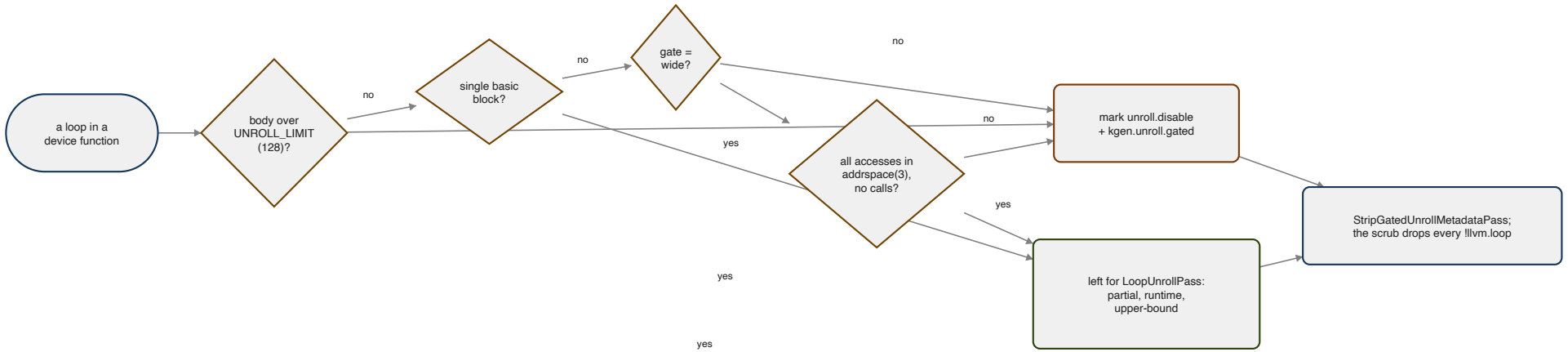
3. Unroll the loops the source left rolled, and nothing else

With SLP off, the FMA-chain bench told the next story. A chain is a dependent sequence of FMAs; more independent chains in flight hide latency. The port's curve from 1 to 32 chains ran 412 → 2,985 GFLOP/s against Apple's 677 → 3,203 from MSL — behind at every count, and the gap was largest where the source had one loop and Apple's compiler unrolled it; neither Mojo pipeline unrolled anything. There was no loop unroller in the device pipeline at all (D24).

Adding LLVM's partial unroller unconditionally was a mixed result:

GFLOP/s, 2048³; FMA chains 1/4/16/64	Unroller off	Unroller on, no gate
register matmul, rolled K-step	958	**997**
SRAM matmul	341	**397**
FMA chains	411 / 1,311 / 2,629 / 3,210	**1,922 / 3,375** / 2,621 / 3,333
register matmul, `comptime for` ×16	973	803
register matmul, ×4 in a loop of 4	1,005	851

The wins are where loops were rolled; the losses are where the source had already unrolled. The kept AIR says what happened to the ×16 variant: the unroller fully unrolled an inner loop that was already the tile load, and Apple's compiler then did worse with the larger body. So the unroller ships behind a gate, and the gate is the design:



Single-block loops are the FMA chain and the rolled K-step: a body of arithmetic with no branches. Multi-block loops that touch only threadgroup memory are the SRAM tile loops, admitted only under the `wide` gate, because the first wide rule — "does the loop touch device memory?" — never fired at all: device pointers are still `addrspace(0)` at the point the unroller runs, before legalisation, so the tile loops slipped back in. The rule became *any access outside addrspace(3)*. The thresholds are set through LLVM's option registry with `addOccurrence`, because `cl::opt::setValue` is ignored by the unroll preferences.

GFLOP/s, 2048³; FMA chains 1/4/16/64	Off	Gate: single-block, threshold 1024	Gate: + threadgroup-only multi-block	The same, threshold 150
×16	953	**974**	803	804
×4	1,004	**1,004**	851	851
rolled	957	954	794	**993**
SRAM	352	**395**	403	398
FMA	414 / 1,321 / 2,641 / 3,219	2,815 / 3,544 / **3,602** / 3,332	1,989 / 3,529 / 3,604 / 3,375	2,096 / 3,400 / 2,624 / 3,219

The single-block gate at threshold 1024 is what ships. No bench regresses, SRAM gains 12%, and the FMA-chain curve reaches 3,544 GFLOP/s at four chains and 3,602 at sixteen — the machine's ~3.6 TFLOP/s peak on the 10-core M4 — where four chains had bought 1,321 and even sixty-four had topped out at 3,225. The rolled matmul's +4% in the last column is real and *open*: it needs a per-loop unroll count rather than one threshold, and the wide gate that reaches it costs the ×16 kernel 17%. The corpus sweep — this fork's own GPU corpus, 506 targets against the 91-target August baseline — came back with zero regressions, which is the condition for a device-pipeline change to land at all.

And the leak that made one of those columns read 116 for a day: the gate's `llvm.loop.unroll.disable` marks, left in the module, reached Apple's compiler, which honoured them on loops it would otherwise have unrolled itself. Hence `StripGatedUnrollMetadataPass`, and the scrub's rule that no loop metadata goes to the driver.

4. Threadgroup memory is accounted twice, sometimes

The examples status file records an anomaly that decides whether a tile fits:

> *The lowered BK=8 variants show an exact 2x ratio: test_matmul_kernel_10 declares 8,320 bytes of shared arrays and its pipeline reports 16,640, while the tuned double-buffer kernel declares 12,544 and reports 25,088. Yet the original 128x128x16 double-buffer tile is rejected at pipeline creation with 33,280 bytes, equal to its declared arrays rather than twice them.*

Whether allocation liveness, unswitching, inlining, or AIR metadata explains the discontinuity is unresolved; the recommendation is to retain pre- and post-legalisation AIR for minimal one- and two-array kernels and find out, because *correct accounting may recover deeper K tiles and performance*. The companion recommendation is contractual: emit the expected static threadgroup byte count in the kernel manifest, verify it after pipeline creation, and reject an oversized specialisation at compile time rather than at launch.

What it takes, in one list

- **Measure Metal's floor first**, with no runtime in the way, so the runtime has a number to be compared with rather than a feeling.
- **Never open an encoder a dispatch did not need.** One encoder per batch, a ring of four command buffers, a hash-keyed function cache.
- **Hand Apple scalar code.** Its compiler is the optimiser. SLP and VectorCombine off; O0 is within 0.3% of the best.
- **Unroll only what the source left rolled**, and strip every mark the gate made before the artefact is written.

- **Trust nothing that did not print a line.** Fresh compile cache per experiment, `[air-knobs]` on every knob, `APPLEGPU_KEEP_AIR` and a census of the `.post.ll` before believing a null result.
- **Verify with the suites**, never with the program that motivated the change, and never on a bench number alone.
- **Run under shader validation and the pipeline-state gate**, or the two silent classes — non-residency and generic pointers — stay silent.
- **Keep residency off the dispatch path.** A residency set edited when an allocation is created or destroyed is O(changes); a `useResource:` walk is O(live allocations) per dispatch.

What is still open is short: the per-loop unroll count for the rolled matmul, precise residency through `air.indirect_buffer`, the threadgroup accounting above, and the D23 remainder in the runtime.

7. What to understand

Ten things about this backend are not obvious from watching it work.

1. AIR is a reader, not a target

There is no instruction selection on this side of the boundary. The compiler borrows an arm64 TargetMachine so the optimiser has something to talk to, rewrites the module until it is Apple-shaped, and serialises it for a frozen LLVM fork inside the Metal compiler service. Every hard problem in the port — version skew, silent semantics, unnamed failures — follows from that one fact. NVPTX and AMDGPU never have it; they consume the optimiser's output in-process.

2. The identity is five numbers, and one is inside the triple

Family, AIR version, Metal language version, SDK version, minimum OS. They vary independently, the AIR and Metal versions are properties of the *toolchain* and not the chip, and `_v28` in `air64_v28-apple-macosx26.0.0` is the AIR version stated a second time. One profile builds all of it, and an unverified profile is refused rather than guessed at.

3. Sample the toolchain, never the table

The stdlib's `+metal3_2,+air2_7_0` pairing describes what the *language* pairs with; the file describes what the *toolchain* writes. Every probe in both Apple profiles of the oracle corpus stamps AIR 2.8.0. The conservative choice was tried once, was rejected, and *taught nothing because the error named nothing*.

4. There is no generic address space, and the failure is silent

> *A kernel whose stores are in addrspace(1) and whose loads are in addrspace(0) writes correctly and reads zero.*

Any `ptr` without an `addrspace` in a finished kernel is a defect unless it is alloca-derived. The idiom for reaching a raw address is `inttoptr`, not `addrspacecast`, which Apple never emits and which AMD's backend needed for a reason that did not survive the port. Shader validation on and off, in one run, tells non-residency from a generic pointer.

5. Inline first, then legalise, then legalise again

Three separate defects were "the code moved after I legalised it". Helpers are inlined before the first AIR-specific rewrite, and the address-space, three-way-compare and deviceize passes run a second time after the post-optimisation inliner, because *on AIR that is not a missed optimisation, it is a wrong answer*.

6. Verify canonical IR; the disassembler lies; the bitstream does not

Gate 1 sits after legalisation and before the typed-pointer downgrade, or it cries wolf twenty-three times for four findings. `llvm-dis` re-populates intrinsic attributes on load, so what it prints is not what was written; dump the module from inside the compiler and read records with `llvm-bcanalyzer`. And the one typed-pointer rule: a call's explicit type must equal the pointee type of its callee — for *every* callee, not the one the fix was first hit on.

7. Form is not deployability

Three gates check form. A module can pass all of them and kill the compiler service at pipeline creation with a message that names nothing — for a dead `declare`, a vector `llvm.fma`, an `i4`. Gate 4 compiles every function in the metallib to a pipeline state. It is twenty lines and belongs in CI.

8. The best optimisation was turning one off

An Apple lane is scalar. SLP packed a fully unrolled matmul sixteen wide and Apple's compiler ran it 9% slower than the scalar form. With SLP and VectorCombine off the port sits at 970 GFLOP/s against the release's 956 and Apple's 951; device O0 is within 0.3% of the best. Apple's compiler is the optimiser. The device pipeline's job is to not get in its way — and to unroll only the loops the source left rolled, behind a gate whose marks are stripped before the driver sees them.

9. Dispatch cost is an encoder boundary

Metal charges 3.5 µs for a fresh compute encoder and 1.0 µs for a dispatch inside an open one, because a boundary is a GPU-side drain. One encoder per batch, a ring of four command buffers and a hash-keyed function cache put the runtime at 0.9–1.0 µs, Metal's own floor, and took STREAM from 81 to 97 GB/s without touching a kernel. The 150 µs round trip for a host-observed result is the same for everyone and is the reason the examples batch a frame's dispatches rather than reading anything back mid-frame.

10. A null experiment is a cache hit until proven otherwise

Six knob experiments measured the same number and emitted identical bitcode because the compile cache had served all six. A silent hit is indistinguishable from "the knob had no effect", so it produces confident false refutations — and it can hide a compiler regression from the suites. Fresh `MODULAR_CACHE_DIR` for anything that exercises the compiler, an `[air-knobs]` line from every knob, retained artefacts before belief, and the suites rather than the motivating program before a push.